

Industrial experiences in the use of static analysis

Zoltán Porkoláb

ELTE Eötvös Loránd University, Faculty of Informatics
Department of Programming Languages and Compilers

gsd@inf.elte.hu
zoltan.porkolab@ericsson.com
<https://gsd.web.elte.hu>

Introduction



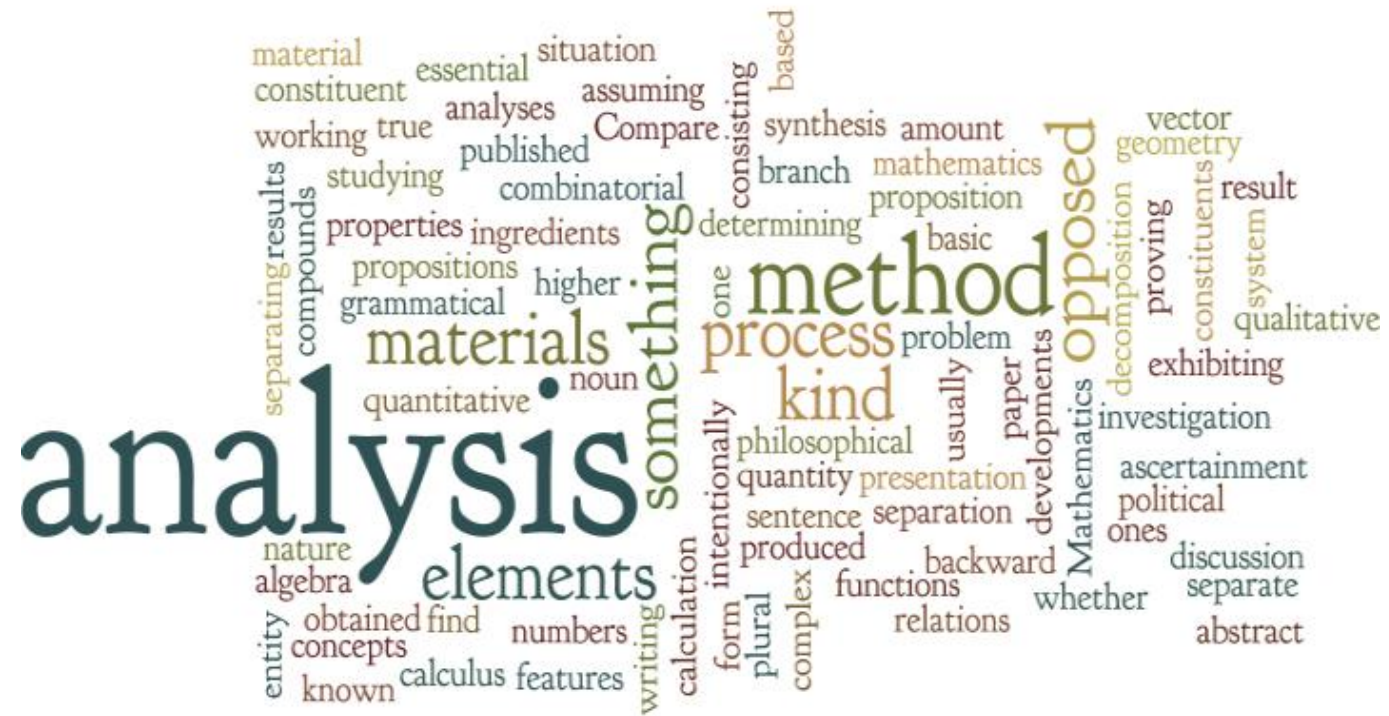
Introduction



Static analysis

Analyzing the source without executing it

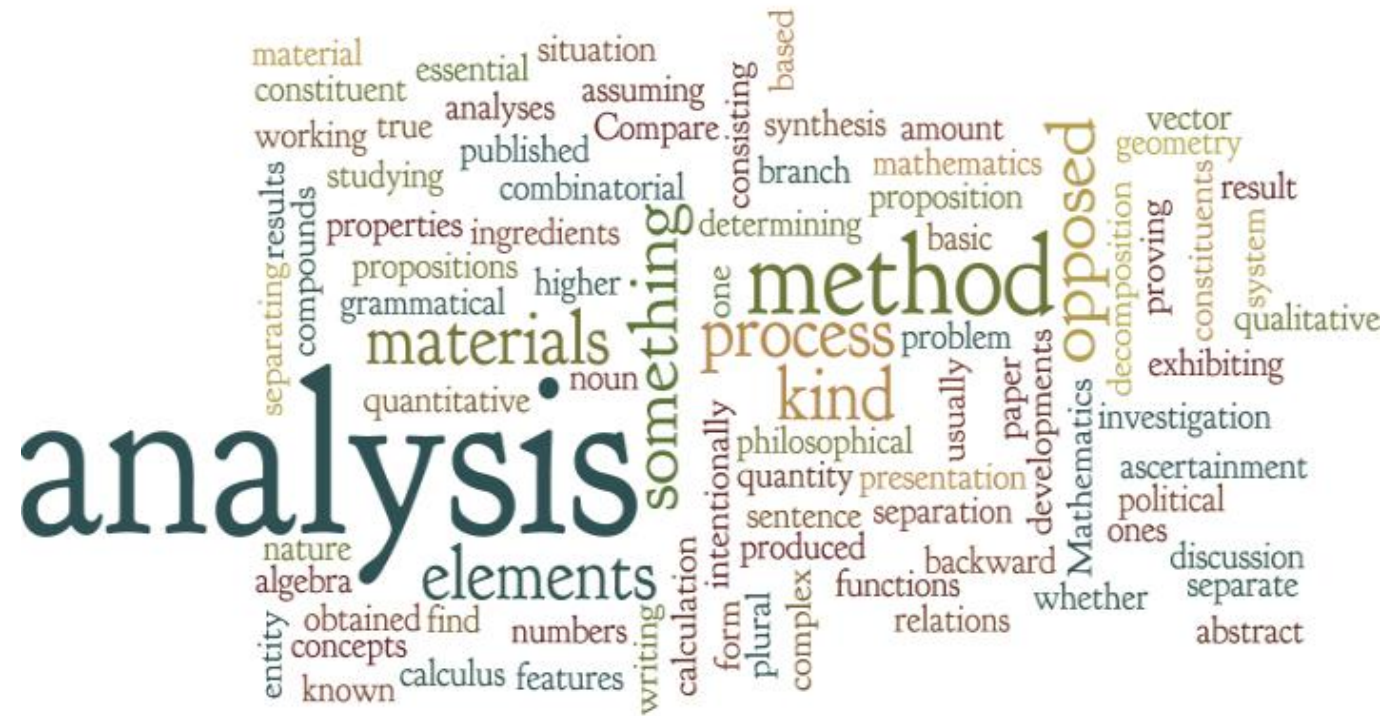
- Finding potential errors
- Optimization
- Automatic code refactoring
- Software metrics
- Code comprehension
- Many other tasks...



Static analysis

Analyzing the source without executing it

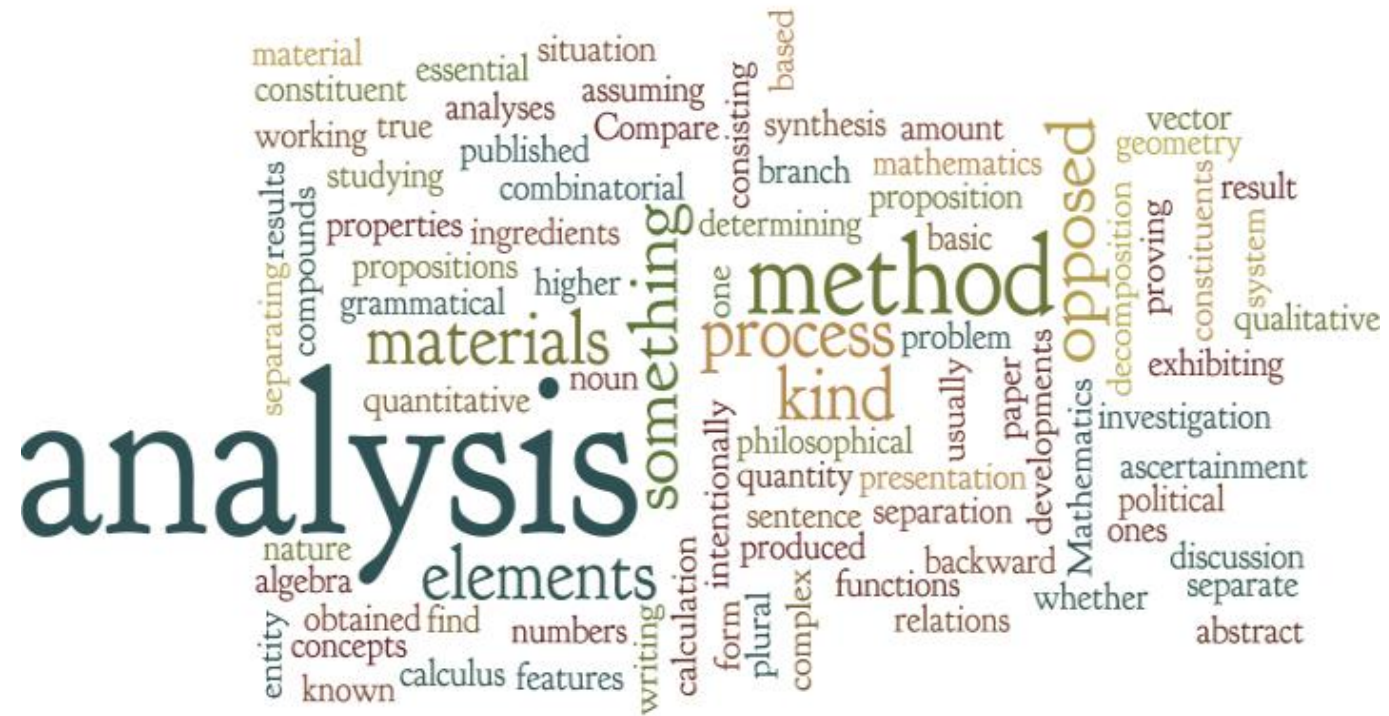
- Finding potential errors
- Optimization
- Automatic code refactoring
- Software metrics
- Code comprehension
- Many other tasks...



Static analysis

Analyzing the source without executing it

- Finding potential errors
- Optimization
- Automatic code refactoring
- Software metrics
- Code comprehension
- Many other tasks...



Comprehension is a major cost factor

Research	Effort for comprehension
IBM (Corbi, 1989)	Over 50% of time
Bell Labs (Davison, 1992)	New project members: 60-80% of time, drops to 20% as one gains experience
National Research Council in Canada (Singer, 2006)	Over 25% of time either searching for or looking at code
Microsoft (Hallam, 2006)	Equal amount of time as design, test
Microsoft (La Toza, 2007)	Over 70% of time
Microsoft (Cherubini, 2007)	95%~ significant part of job 65%< at least once a day 25%< multiple times of a day



Comprehension is a major cost factor

Required for

- Maintenance
- New feature development
- Refactoring
- Performance improvement
- Knowledge rump-up



Tool support is required



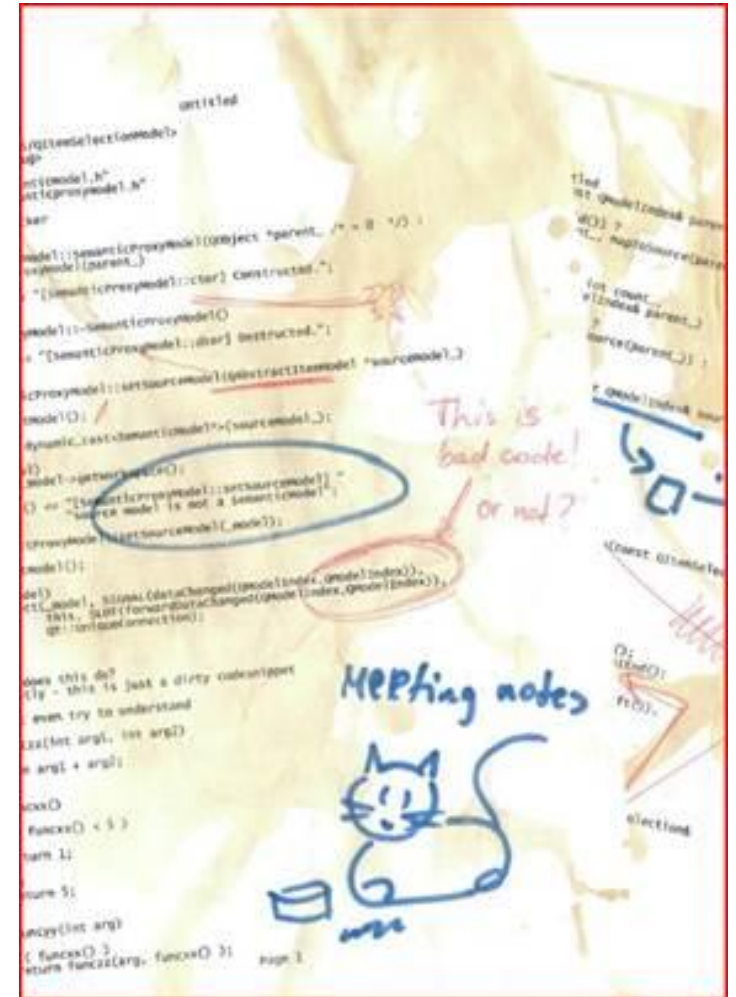
Tool support is required



Tool support is required



Tool support is required



Existing tools

- Easy to use
 - Web based
 - Partial/light parsing
 - OpenGrok
- Deep analysis
 - Parsing with deep C/C++ knowledge
 - Fat client
 - Understand C++
 - CodeSurfer
- Mixture of features
 - Woboq (deep analysis + web)

The screenshot displays the OpenGrok web interface. At the top, the URL is `http://ffmpeg.org/ffmpeg.c`. The search bar contains the text `only in /ffmpeg/src/`. The main content area shows C code from `ffmpeg.c`, with line numbers 3279 to 3310 visible. The code includes a `if (debug_ts)` block and a `if (!list->wrap_correction_done && is->start_time != AV_NOPTS_VALUE && list->st->pts_wrap_bits < 64)` block. A sidebar on the right shows a search results panel with a list of files and their sizes. Below the code, there is a call graph visualization showing the relationships between different parts of the code, with nodes representing functions and edges representing calls. The graph is complex, with many nodes and edges, and is titled 'Call Graph'.

Existing tools

- IDE-based
 - Visual Studio
 - NetBeans
 - Qt Creator
 - Eclipse

OpenGrok

xref: /ffmpeg/src/ffmpeg.c

Home | History | Annotate | Line# | Navigate | Raw | Download Search only in /ffmpeg/src/

```

3279 if (debug_ts) {
3280     av_log(NULL, AV_LOG_INFO, "demuxer -> ist_index: %d type: %s\n",
3281         next_dts: %s next_dts_time: %s next_pts: %s next_pts_time: %s pkt_pts: %s pkt_pts_time: %s pkt_dt
3282         ifile->ist_index + pkt.stream_index, av_get_media_type_string(ist->codec->codec_type),
3283         av_ts2str(ist->next_dts), av_ts2timestr(ist->next_dts, &AV_TIME_BASE_Q),
3284         av_ts2str(ist->next_pts), av_ts2timestr(ist->next_pts, &AV_TIME_BASE_Q),
3285         av_ts2str(pkt.pts), av_ts2timestr(pkt.pts, &ist->st->time_base),
3286         av_ts2str(pkt.dts), av_ts2timestr(pkt.dts, &ist->st->time_base),
3287         av_ts2str(input_files[ist->file_index]->ts_offset),
3288         av_ts2timestr(input_files[ist->file_index]->ts_offset, &AV_TIME_BASE_Q));
3289 }
3290
3291 if(!ist->wrap_correction_done && ist->start_time != AV_NOPTS_VALUE && ist->st->pts_wrap_bits < 64){
3292     int64_t stime, stime2;
3293     // Correcting starttime based on the enabled streams
3294     // FIXME this ideally should be done before the first use of starttime but we do not know which are
3295     // so we instead do it here as part of discontinuity handling
3296     if ( ist->next_dts == AV_NOPTS_VALUE
3297         && ifile->ts_offset == -1 && start_time
3298         && (is->format->flags & AVFMT_TS_DISCONT)) {
3299         int64_t new_start_time = INT64_MAX;
3300         for (l=0; l<ist->n_b_streams; l++) {
3301             AVStream *st = ist->streams[l];
3302             if (st->discard == AVDISCARD_ALL || st->start_time == AV_NOPTS_VALUE)
3303                 continue;
3304             new_start_time = FFMIN(new_start_time, av_rescale_q(st->start_time, st->time_base, AV_TIME
3305         }
3306         if (new_start_time > is->start_time) {
3307             av_log(is, AV_LOG_VERBOSE, "Correcting start time by %\"PRIu64\"\\n", new_start_time - is->start
3308             ifile->ts_offset = -new_start_time;
3309         }
3310     }
3311 }

```

```

76 size_t NewCapacity
77 while (NewCapacity < 0)
78     NewCapacity = grow
79 grow
80 }
81
82 // Because we are traversing elements to the TypeLoc backwards, we have to
83 // do this.
84 // FIXE
85 // need
86 // FIXE
87 // handle
88 if (Local)
89 if (Null)
90 NullBy
91 } else {

```

clang::TypeLocBuilder::grow (4 rw)

clang::TypeLocBuilder::pushImp1 (3 r)

llvm/tools/clang/lib/Sema/TypeLocBuilder.h:31

Show Uses: (12)

- **llvm/tools/clang/lib/Sema/TypeLocBuilder.cpp (7)**
- **clang::TypeLocBuilder::grow (4 rw)**
- **clang::TypeLocBuilder::pushImp1 (3 r)**
- **llvm/tools/clang/lib/Sema/TypeLocBuilder.h (5)**
- **clang::TypeLocBuilder::TypeLocBuilder (1 w)**
- **clang::TypeLocBuilder::reserve (1 r)**
- **clang::TypeLocBuilder::clear (1 r)**
- **clang::TypeLocBuilder::getTypeSourceInfo (1 r)**
- **clang::TypeLocBuilder::getTypeLocInContext (1 r)**

clang::TypeLocBuilder::clear (1 r)

clang::TypeLocBuilder::getTypeSourceInfo (1 r)

clang::TypeLocBuilder::getTypeLocInContext (1 r)

Comprehension requires specific feature list

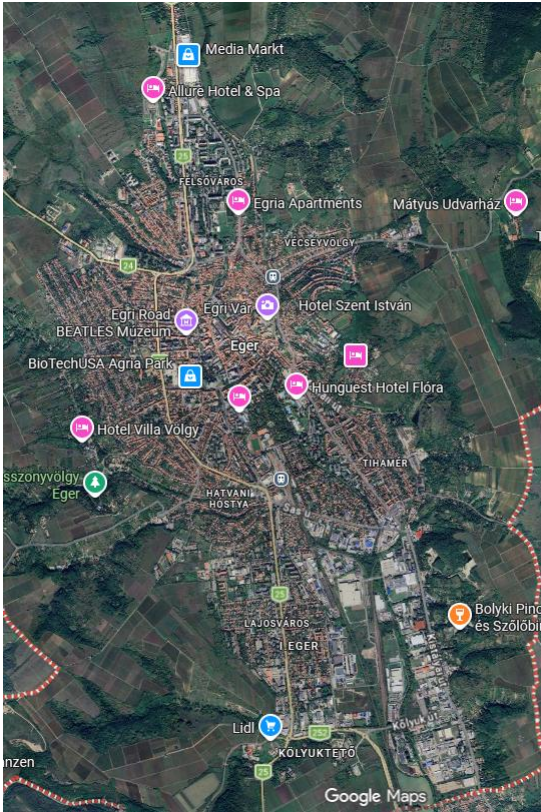
Development of code	Understanding code
Writing new code (support: code completion, etc.)	Reading and navigating inside code
Intentions are clear	Intensions are weak
Working with only a few files at the same time	Frequently jumping between many different files
Edit, compile, fix, repeat	Visualize
Working on the same abstraction level for a while	Jumping between various abstraction levels (Google map of code)



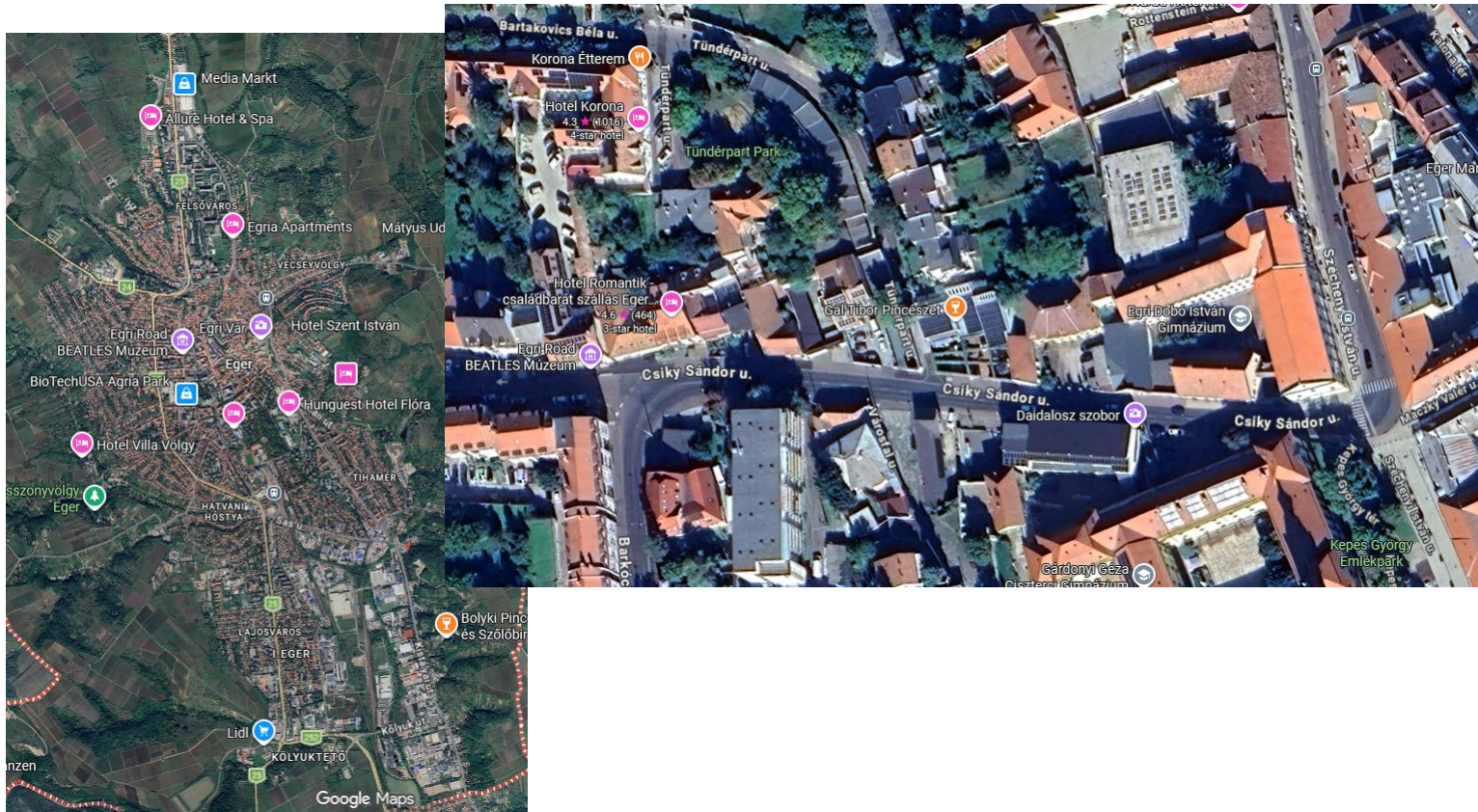
Comprehension requires specific feature list



Comprehension requires specific feature list



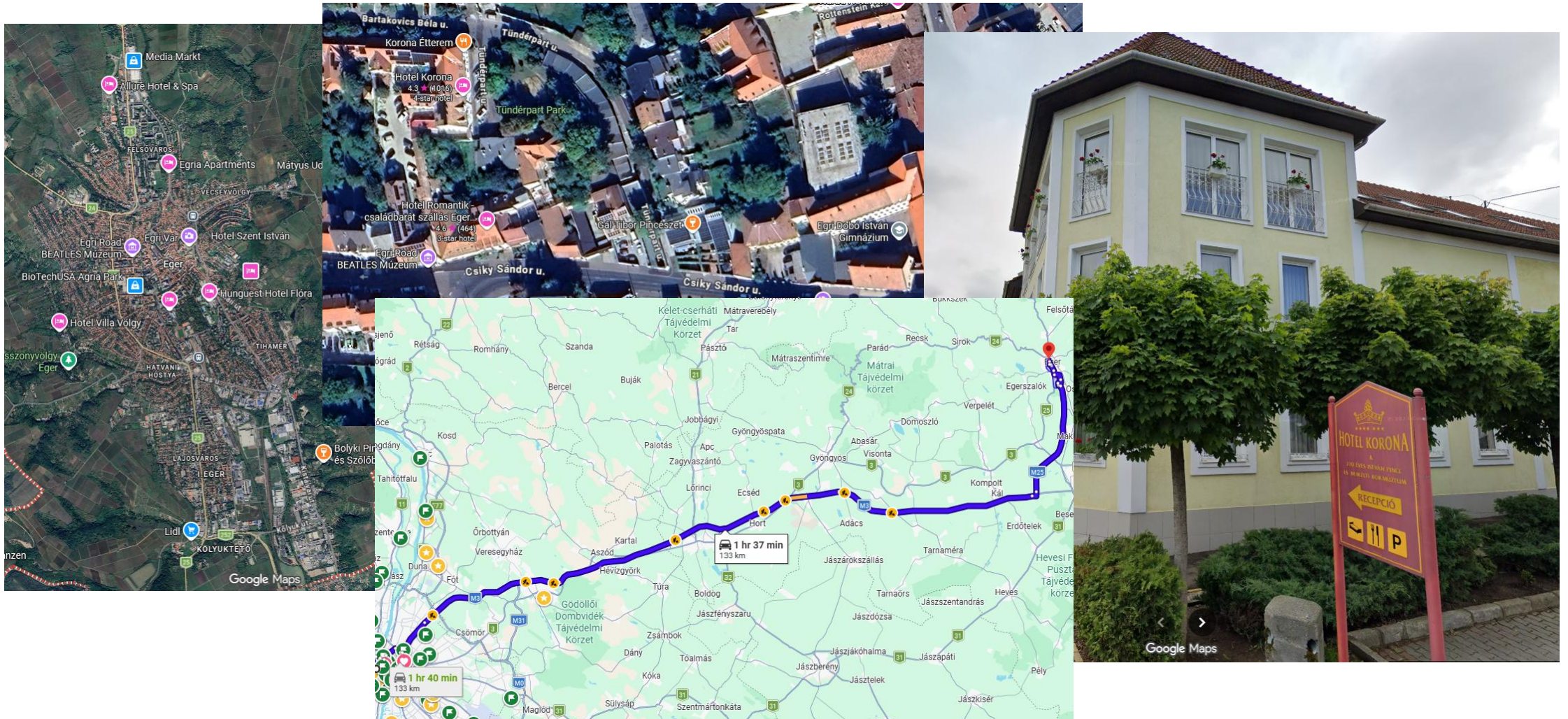
Comprehension requires specific feature list



Comprehension requires specific feature list



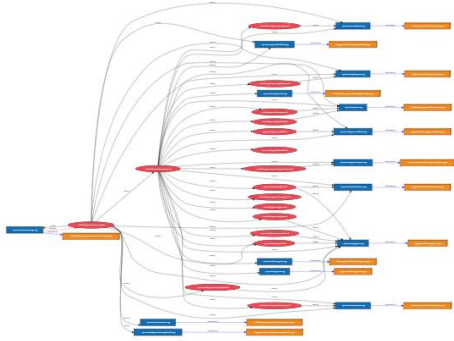
Comprehension requires specific feature list



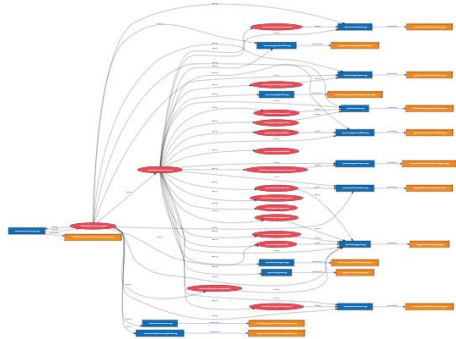
Comprehension requires specific feature list



Comprehension requires specific feature list



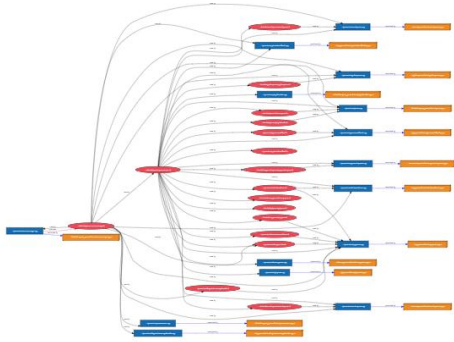
Comprehension requires specific feature list



```
90
91 /**
92  * This function checks the existence of the workspace and project directory
93  * based on the given command line arguments.
94  * @return whether the project directory exists or not.
95  */
96 bool checkProjectDir(const pcv::variables_map& vm_)
97 {
98     const std::string projDir
99     = vm_["workspace"].as<std::string>() + '/'
100     + vm_["name"].as<std::string>();
101     return fs::is_directory(projDir);
102 }
103
104 /**
105  * This function prepares the workspace and project directory based on the given
106  * command line arguments. If the project directory can't be created because of
107  * permission issues then empty string returns which indicates the problem.
108  * @return The path of the project directory.
109  */
110 std::string prepareProjectDir(const pcv::variables_map& vm_)
111 {
112     const std::string projDir
113     = vm_["workspace"].as<std::string>() + '/'
114     + vm_["name"].as<std::string>();
115     boost::system::error_code ec;
116     bool isDirProject = fs::create_directories(projDir, ec);
117     if (ec)
118     {
119         LOG(error) << "Permission denied to create " + projDir;
120         return std::string();
121     }
122     if (!isDirProject)
123         return projDir;
124     if (vm_.count("force"))
125     {
126         fs::remove_all(projDir, ec);
127         if (ec)
128         {
129             LOG(error) << "Permission denied to remove " + projDir;
130             return std::string();
131         }
132         fs::create_directory(projDir, ec);
133         if (ec)
134         {
135             LOG(error) << "Permission denied to create " + projDir;
136             return std::string();
137         }
138     }
139     return projDir;
140 }
```



Comprehension requires specific feature list



```
106
107
108 // * This function checks the existence of the workspace and project directory
109 // * based on the given command line arguments.
110 // * Return whether the project directory exists or not.
111
112 bool checkProjectDir(const po::variables_map& vm_)
113 {
114     const std::string projDir
115         = vm_["workspace"].as<std::string>() + '/'
116         + vm_["name"].as<std::string>();
117     return fs::is_directory(projDir);
118 }
119
120
121 // * This function prepares the workspace and project directory based on the given
122 // * command line arguments. If the project directory can't be created because of
123 // * permission issues then empty string returns which indicates the problem.
124 // * Return the path of the project directory.
125 std::string prepareProjectDir(const po::variables_map& vm_)
126 {
127     const std::string projDir
128         = vm_["workspace"].as<std::string>() + '/'
129         + vm_["name"].as<std::string>();
130     boost::system::error_code ec;
131     bool isNewProject = fs::create_directories(projDir, ec);
132     if (ec)
133     {
134         LOG(error) << "Permission denied to create " + projDir;
135         return std::string();
136     }
137     if (isNewProject)
138     {
139         if (vm_.count("force"))
140         {
141             fs::remove_all(projDir, ec);
142             if (ec)
143             {
144                 LOG(error) << "Permission denied to remove " + projDir;
145                 return std::string();
146             }
147             fs::create_directory(projDir, ec);
148             if (ec)
149             {
150                 LOG(error) << "Permission denied to create " + projDir;
151                 return std::string();
152             }
153         }
154     }
155     return projDir;
156 }
```

```
(fully parsed) parser.cpp :: /projects/CodeCompass/parser/src/parser.
117
118 p
119
120 std::string prepareProjectDir(const po::variables_map& vm_)
121 {
122     const std::string projDir
123         = vm_["workspace"].as<std::string>() + '/'
124         + vm_["name"].as<std::string>();
125     boost::system::error_code ec;
126     bool isNewProject = fs::create_directories(projDir, ec);
127     if (ec)
128     {
129         LOG(error) << "Permission denied to create " + projDir;
130         return std::string();
131     }
132     if (isNewProject)
133         return projDir;
134 }
135
```

boost::filesystem::create_directories(class BOOST_PROGRAM_OPTIONS_DECL variables_map : public abstract_variables_map, public std::map {})

```
(fully parsed) operations.hpp :: /usr/include/boost/filesystem/operat
ns.hpp
373
374 inline
375 bool create_directories(const path& p, system::error_code& ec) BOOST
{return detail::create_director
```

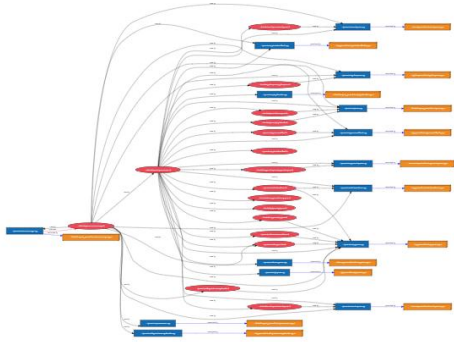
```
(fully parsed) variables_map.hpp :: /usr/include/boost/program_option
variables_map.hpp
146
147 class BOOST_PROGRAM_OPTIONS_DECL variables_map : public abstract
public std::map<std::string, vari
148
149 {
150     variables_map();
151     variables_map(const abstract_variables_map& next);
152
153     // Resolve conflict between inherited operators.
154     const variable_value& operator[] (const std::string& name)
155     { return abstract_variables_map::operator[] (name); }
156
157     // Override to clear some extra fields.
158     void clear();
159     void notify();
160
161 private:
162     // Implementation of abstract_variables_map
163
164
```

class BOOST_PROGRAM_OPTIONS_DECL abstract_variables_map {}

```
(fully parsed) variables_map.hpp :: /usr/include/boost/program_option
variables_map.hpp
106
107 class BOOST_PROGRAM_OPTIONS_DECL abstract_variables_map {
108 public:
109     abstract_variables_map();
110     abstract_variables_map(const abstract_variables_map& next)
111     virtual ~abstract_variables_map() {}
112
113     /** Obtains the value of variable 'name', from *this and
114     possibly from the chain of variable maps.
115
116     - if there's no value in *this.
117     - if there's next variable map, returns value from
118     - otherwise, returns empty value
119
120     - if there's defaulted value
121     - if there's next variable map, which has a non-
122     value, return that
123     otherwise, return value from this
124
```



Comprehension requires specific feature list



```
98 /**
99  * This function checks the existence of the workspace and project directory
100  * based on the given command line arguments.
101  * @return Whether the project directory exists or not.
102  */
103 bool checkProjectDir(const po::variables_map& vm_)
104 {
105     const std::string projDir
106     = vm_["workspace"].as<std::string>() + '/'
107     + vm_["name"].as<std::string>();
108     return fs::is_directory(projDir);
109 }
110 /**
111  * This function prepares the workspace and project directory based on the given
112  * command line arguments. If the project directory can't be created because of
113  * permission issues then empty string returns which indicates the problem.
114  * @return The path of the project directory.
115  */
116 std::string prepareProjectDir(const po::variables_map& vm_)
117 {
118     const std::string projDir
119     = vm_["workspace"].as<std::string>() + '/'
120     + vm_["name"].as<std::string>();
121     boost::system::error_code ec;
122     bool isNewProject = fs::create_directories(projDir, ec);
123     if (ec)
124     {
125         LOG(error) << "Permission denied to create " + projDir;
126         return std::string();
127     }
128     if (isNewProject)
129     {
130         if (vm_.count("force"))
131         {
132             fs::remove_all(projDir, ec);
133             if (ec)
134             {
135                 LOG(error) << "Permission denied to remove " + projDir;
136                 return std::string();
137             }
138         }
139     }
140     return projDir;
141 }
```

File Manager ..\CodeCompass\parser\src\parser.cpp

Search Results

Info Tree

- Function: std::string prepareProjectDir(const po::variables_map& vm_)
- Name: prepareProjectDir
- Qualified name: prepareProjectDir
- Signature: prepareProjectDir
- Callee(21)
- Caller(1)
- parser.cpp(1)
- 220:1: main : int
- Definition(1)
- 117:1: prepareProjectDir : std::string
- Local variables(6)
- Parameters(1)
- 117:31: const po::variables_map& vm
- This calls(60)
- Usage(2)

Revision Control Navigator

```
(Fully parsed) parser.cpp :: /projects/CodeCompass/parser/src/parser.cpp
95 }
96
97 /**
98  * This function checks the existence of the workspace and project direc
99  * based on the given command line arguments.
100  * @return Whether the project directory exists or not.
101  */
102 bool checkProjectDir(const po::variables_map& vm_)
103 {
104     const std::string projDir
105     = vm_["workspace"].as<std::string>() + '/'
106     + vm_["name"].as<std::string>();
107     return fs::is_directory(projDir);
108 }
109
110 /**
111  * This function prepares the workspace and project directory based on t
112  * command line arguments. If the project directory can't be created be
113  * permission issues then empty string returns which indicates the prob
114  * @return The path of the project directory.
115  */
116 std::string prepareProjectDir(const po::variables_map& vm_)
117 {
118     const std::string projDir
119     = vm_["workspace"].as<std::string>() + '/'
120     + vm_["name"].as<std::string>();
121
122 }
```

```
(Fully parsed) parser.cpp :: /projects/CodeCompass/parser/src/parser.cpp
117 std::string prepareProjectDir(const po::variables_map& vm_)
118 {
119     const std::string projDir
120     = vm_["workspace"].as<std::string>() + '/'
121     + vm_["name"].as<std::string>();
122     boost::system::error_code ec;
123     bool isNewProject = fs::create_directories(projDir, ec);
124     if (ec)
125     {
126         LOG(error) << "Permission denied to create " + projDir;
127         return std::string();
128     }
129     if (isNewProject)
130     {
131         return projDir;
132     }
133 }
```

boost::filesystem::create_directories(class BOOST_PROGRAM_OPTIONS_DECL variables_map : public abstract_variables_map, public std::map<string, variable_value> & next)

```
(Fully parsed) operations.hpp :: /usr/include/boost/filesystem/operations.hpp
373 inline
374 bool create_directories(const path& p, system::error_code& ec) BOOST_NOEXCEPT
375 {
376     return detail::create_directories(p, ec);
377 }
```

```
(Fully parsed) variables_map.hpp :: /usr/include/boost/program_options/variables_map.hpp
146 class BOOST_PROGRAM_OPTIONS_DECL variables_map : public abstract_variables_map {
147 public:
148     variables_map();
149     variables_map(const abstract_variables_map& next);
150     // Resolve conflict between inherited operators.
151     const variable_value& operator[] (const std::string& name) const {
152         return abstract_variables_map::operator[] (name);
153     }
154     // Override to clear some extra fields.
155     void clear();
156     void notify();
157 private:
158     // Implementation of abstract_variables_map
159 }
```

class BOOST_PROGRAM_OPTIONS_DECL abstract_variables_map {

```
(Fully parsed) variables_map.hpp :: /usr/include/boost/program_options/variables_map.hpp
180 class BOOST_PROGRAM_OPTIONS_DECL abstract_variables_map {
181 public:
182     abstract_variables_map();
183     abstract_variables_map(const abstract_variables_map& next);
184     virtual ~abstract_variables_map() {}
185     /** Obtains the value of variable 'name', from *this and
186     * possibly from the chain of variable maps.
187     * - if there's no value in *this,
188     *   - if there's next variable map, returns value from it
189     *   - otherwise, returns empty value
190     * - if there's defaulted value
191     *   - if there's next variable map, which has a non-empty
192     *     value, return that
193     *   - otherwise, return value from *this
194     */
195     const variable_value& operator[] (const std::string& name) const {
196         return get_value(name);
197     }
198     void clear();
199     void notify();
200 }
```



Comprehension requires specific feature list

- Deep analysis + build information
- Fast text- based feature location
- Architectural information
- Textual summaries
- (Interactive) Visualizations
- Working on the whole knowledge portfolio
- Scalability (>10 millions LOC)
- Fast user interface (<1sec)
- Communication between developers (permalinks)
- Extendibility: open-source platform

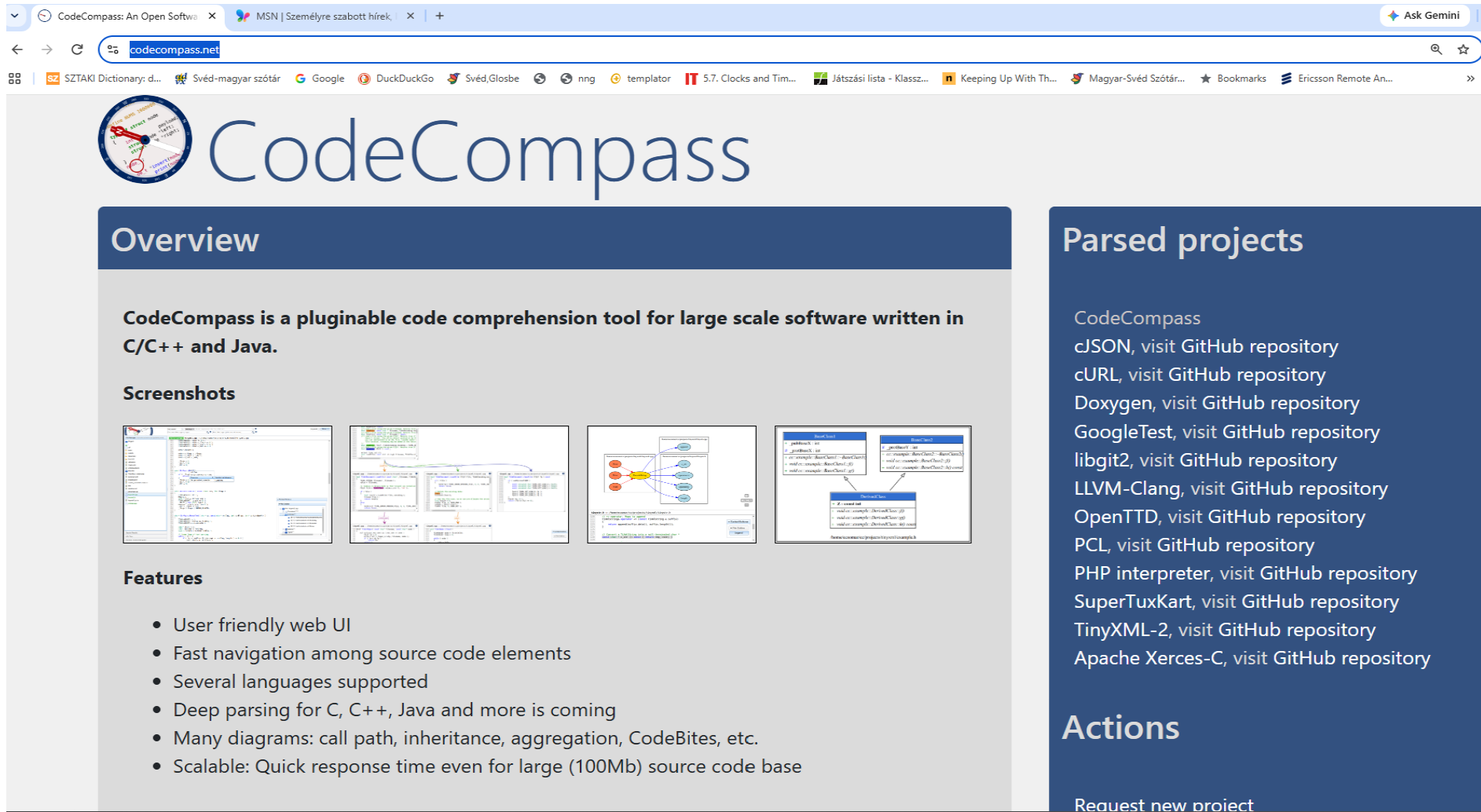


CodeCompass: an open-source comprehension framework

- Joint work of Eötvös Loránd University and Ericsson Hungary
- Started at 2012
- Open source since 2017 <https://github.com/Ericsson/CodeCompass>
- Used in Ericsson and outside
- Supports C/C++, Python, C#
- Experimental: Java, Haskell
- Trial page with parsed projects: <https://codecompass.net/>



CodeCompass trial page



CodeCompass

Overview

CodeCompass is a pluginable code comprehension tool for large scale software written in C/C++ and Java.

Screenshots



Features

- User friendly web UI
- Fast navigation among source code elements
- Several languages supported
- Deep parsing for C, C++, Java and more is coming
- Many diagrams: call path, inheritance, aggregation, CodeBites, etc.
- Scalable: Quick response time even for large (100Mb) source code base

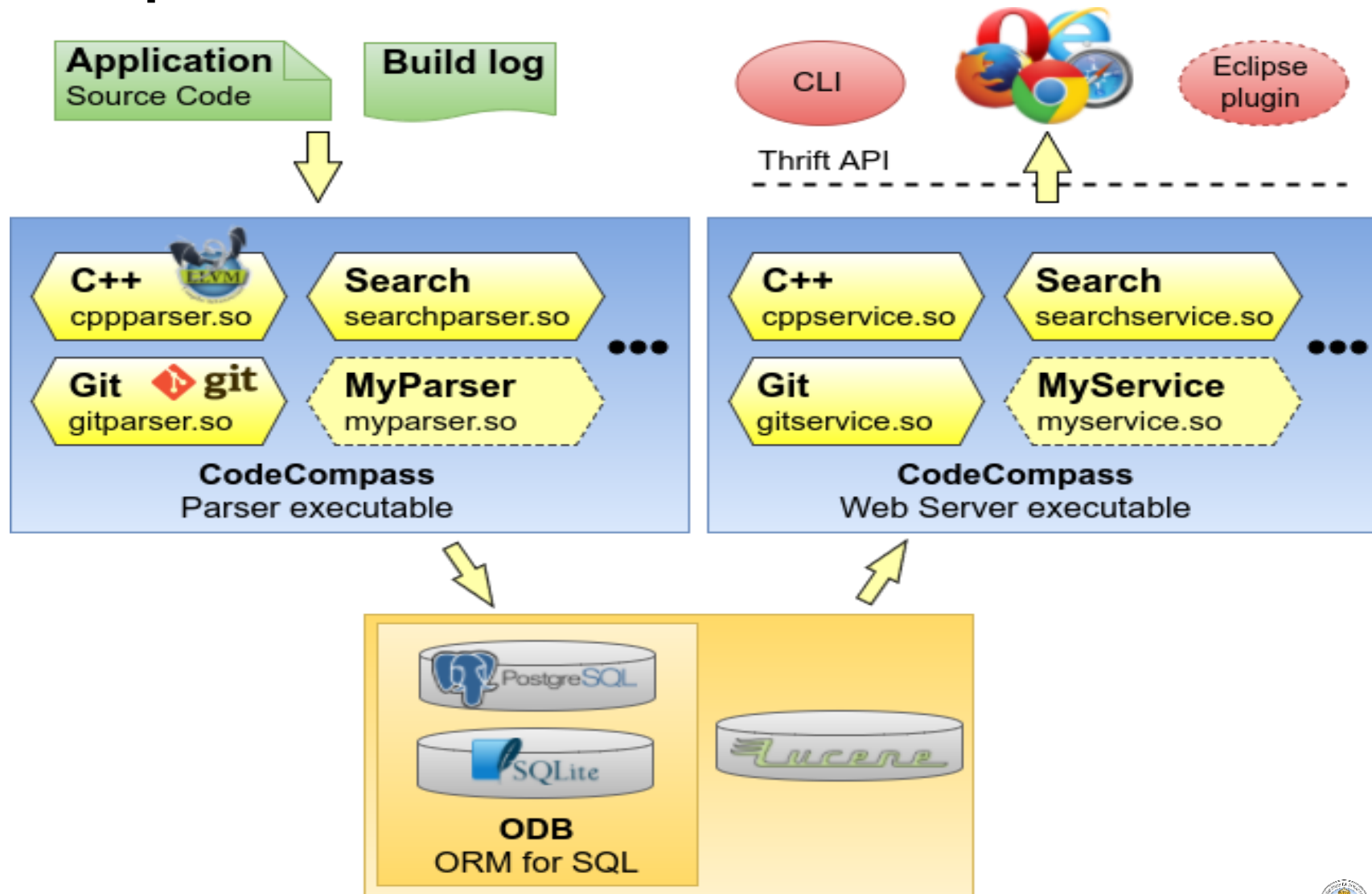
Parsed projects

CodeCompass
cJSON, visit [GitHub repository](#)
cURL, visit [GitHub repository](#)
Doxygen, visit [GitHub repository](#)
GoogleTest, visit [GitHub repository](#)
libgit2, visit [GitHub repository](#)
LLVM-Clang, visit [GitHub repository](#)
OpenTTD, visit [GitHub repository](#)
PCL, visit [GitHub repository](#)
PHP interpreter, visit [GitHub repository](#)
SuperTuxKart, visit [GitHub repository](#)
TinyXML-2, visit [GitHub repository](#)
Apache Xerces-C, visit [GitHub repository](#)

Actions

[Reaquest new project](#)

CodeCompass architecture



Design choice: storage

Store the Abstract Syntax Tree

Produced by Clang

All information is there

Problem: size

1:200-500 ratio between source and AST

Easily grown over 50GB + indexes

Not scalable

Reason

Header duplications

Design choice: storage

- Store only named entities
 - Class/function/variable declarations
 - References to class/function/variable usage
 - Source as text
 - Build information
- Scalable
 - 1:30-50 ratio between source and AST
 - Object-relational mapping with ODB <https://www.codesynthesis.com/products/odb/>
 - Full LLVM is stored in 13 GB with indexes in Postgres
 - Fast text based search elements in Lucene DB
- We loose information
 - Reparse

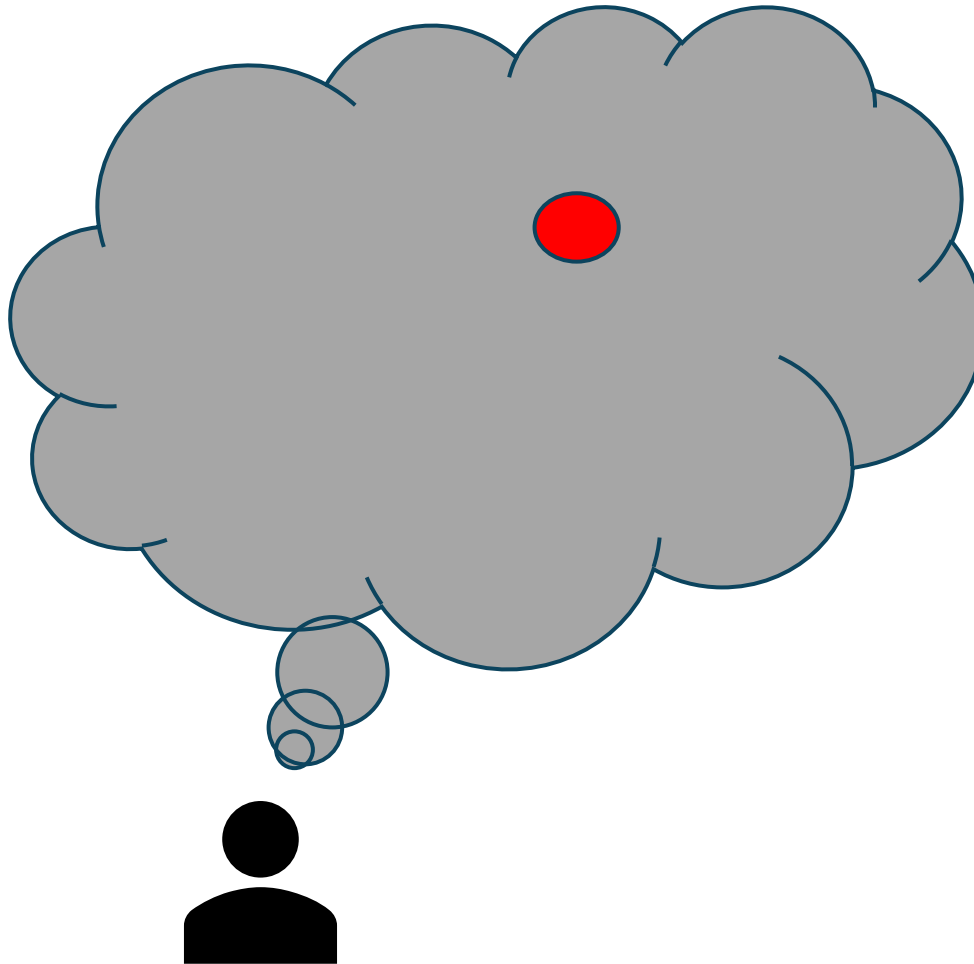


Typical workflow

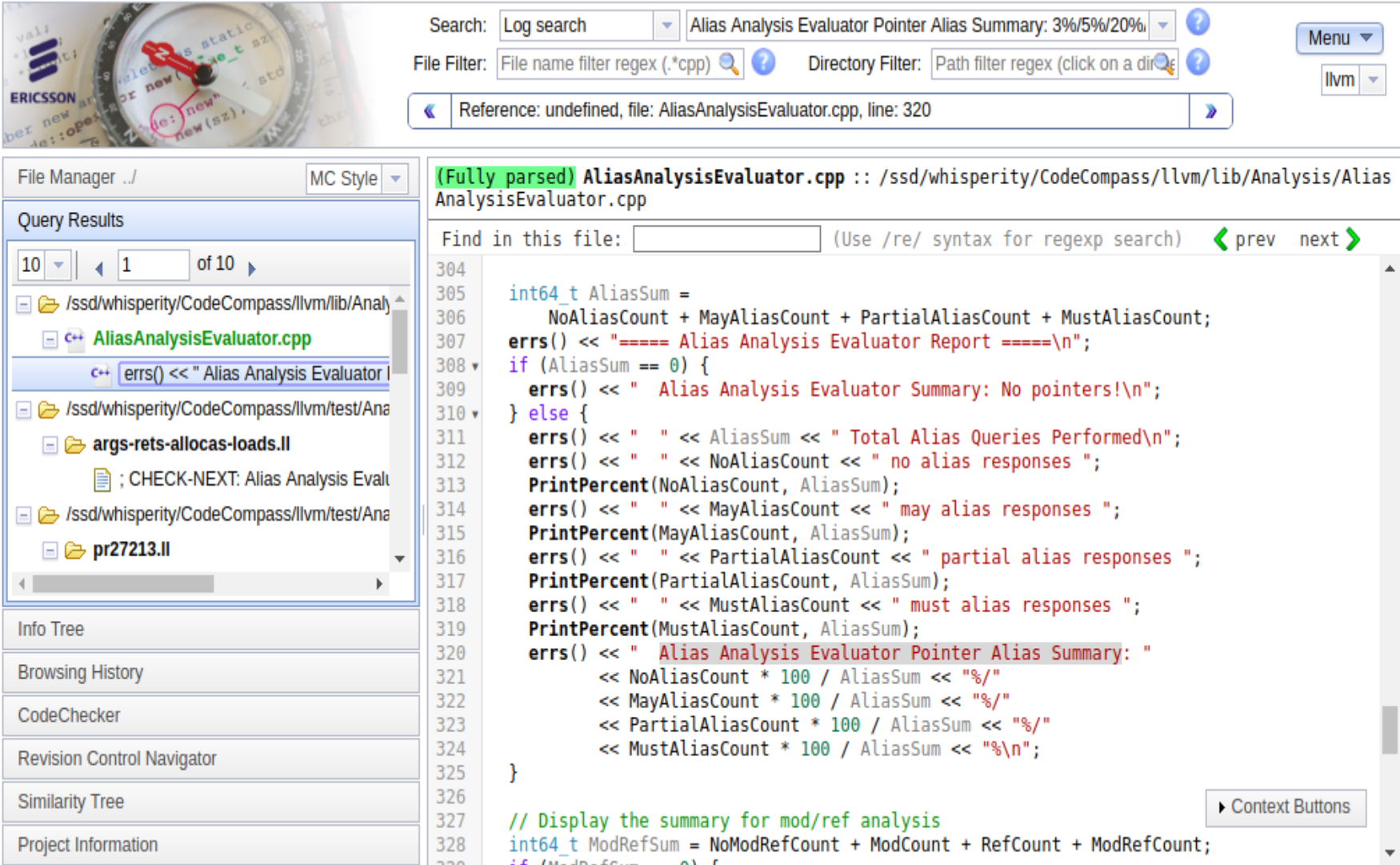


Typical workflow

- Fast feature location
 - Textual search
 - Log-base search



Fast feature location / log search



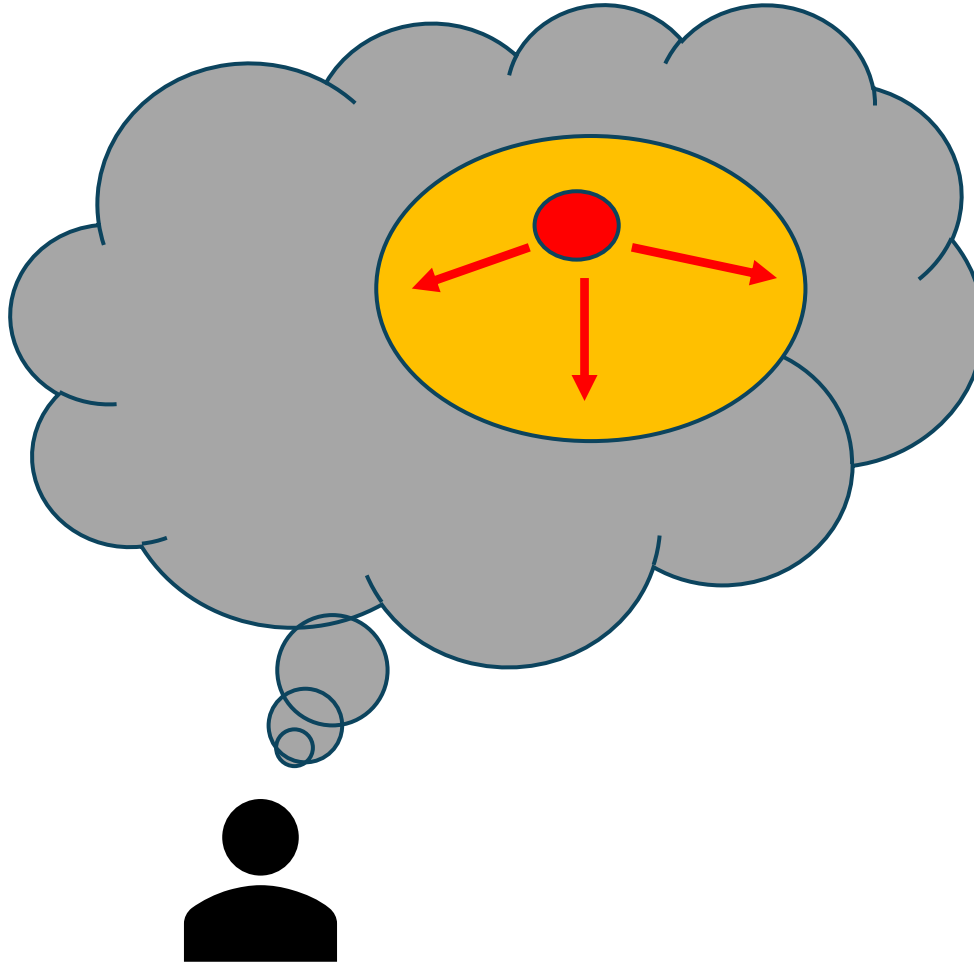
The screenshot displays the CodeCompass IDE interface. At the top, the search bar is set to "Log search" and the file filter is "File name filter regex (*.cpp)". The directory filter is "Path filter regex (click on a dir)". The search results show a reference to "undefined, file: AliasAnalysisEvaluator.cpp, line: 320".

The left sidebar shows the "Query Results" panel with a list of files. The file "AliasAnalysisEvaluator.cpp" is selected. Below it, the "Info Tree" panel shows the file's location: "/ssd/whisperity/CodeCompass/llvm/lib/Analysis/AliasAnalysisEvaluator.cpp".

The main editor window displays the source code of "AliasAnalysisEvaluator.cpp". The code is highlighted in green, indicating it is the selected file. The code shows the "Alias Analysis Evaluator" logic, including the "AliasSum" variable and the "Alias Analysis Evaluator Summary" output.

```
304 int64_t AliasSum =
305     NoAliasCount + MayAliasCount + PartialAliasCount + MustAliasCount;
306 errs() << "==== Alias Analysis Evaluator Report =====\n";
307 if (AliasSum == 0) {
308     errs() << " Alias Analysis Evaluator Summary: No pointers!\n";
309 } else {
310     errs() << " " << AliasSum << " Total Alias Queries Performed\n";
311     errs() << " " << NoAliasCount << " no alias responses ";
312     PrintPercent(NoAliasCount, AliasSum);
313     errs() << " " << MayAliasCount << " may alias responses ";
314     PrintPercent(MayAliasCount, AliasSum);
315     errs() << " " << PartialAliasCount << " partial alias responses ";
316     PrintPercent(PartialAliasCount, AliasSum);
317     errs() << " " << MustAliasCount << " must alias responses ";
318     PrintPercent(MustAliasCount, AliasSum);
319     errs() << " Alias Analysis Evaluator Pointer Alias Summary: "
320         << NoAliasCount * 100 / AliasSum << "%/"
321         << MayAliasCount * 100 / AliasSum << "%/"
322         << PartialAliasCount * 100 / AliasSum << "%/"
323         << MustAliasCount * 100 / AliasSum << "%\n";
324 }
325
326 // Display the summary for mod/ref analysis
327 int64_t ModRefSum = NoModRefCount + ModCount + RefCount + ModRefCount;
328 if (ModRefSum == 0) {
```

Typical workflow



- Fast feature location
 - Textual search
 - Log-base search
- **Extend knowledge**
 - Info tree
 - CodeBites view
 - Function call chains
 - Browsing history

Easy code browsing -- CodeBites

Architectural information

The screenshot displays the CodeCompass web interface for a project named 'compass'. The top navigation bar includes a search bar, file and directory filters, and a menu. The left sidebar shows a file manager with a list of files and folders. The main area features a dependency diagram with nodes representing source files and libraries, connected by arrows indicating relationships like 'provides', 'used by', and 'contained by'. The bottom section shows the source code for 'ccfrontendaction.h'.

Search: Text search
File Filter: File name filter regex (*.cpp)
Directory Filter: Path filter regex (click on a dir below)

Project: compass
Jump to sourcedir
...
.libs
assignmentcollecto
ccastconsumer.cpp
ccastconsumer.h
ccfrontendaction.cpp
ccfrontendaction.h
ccppcallbacks.cpp
ccppcallbacks.h
clangastvisitor.h
cxxastpersister.cpp
cxxparser.cpp
cxxparsesession.h
declcontextvisitor.h
documentationcom

Query Results
Info Tree
Browsing History
MI Navigator
Revision Control
Navigator
Similarity Tree
Project Information

Diagram labels:
.../cxxparser/src/ccfrontendaction.h
.../cxxparser/src/cxxparser.cpp
.../cxxparser/src/lib_libcxxparser_la-cxxparser.o
.../src/.libs/lib_libcxxparser_la-cxxparser.o
.../cxxparser/src/ccfrontendaction.cpp
.../src/.libs/lib_libcxxparser_la-ccfrontendaction.o
.../cxxparser/src/lib_libcxxparser_la-ccfrontendaction.o

Context Buttons:
File Outline
File diagrams
Metrics
Text View
Export SVG
Legend
Non-reduced view

(Fully parsed) ccfrontendaction.h :: /ssd/whisperity/CodeCompass_Discovery/ccSrcMaster/parser/cxxparser/src/ccfrontendaction.h

```
1 /*  
2  * macrofrontendaction.h  
3  *  
4  * Created on: Jul 4, 2013  
5  * Author: ezoltbo  
6  */  
7  
8 #ifndef CODECOMPASS_MACROFRONTENDACTION_H_  
9 #define CODECOMPASS_MACROFRONTENDACTION_H_  
10
```

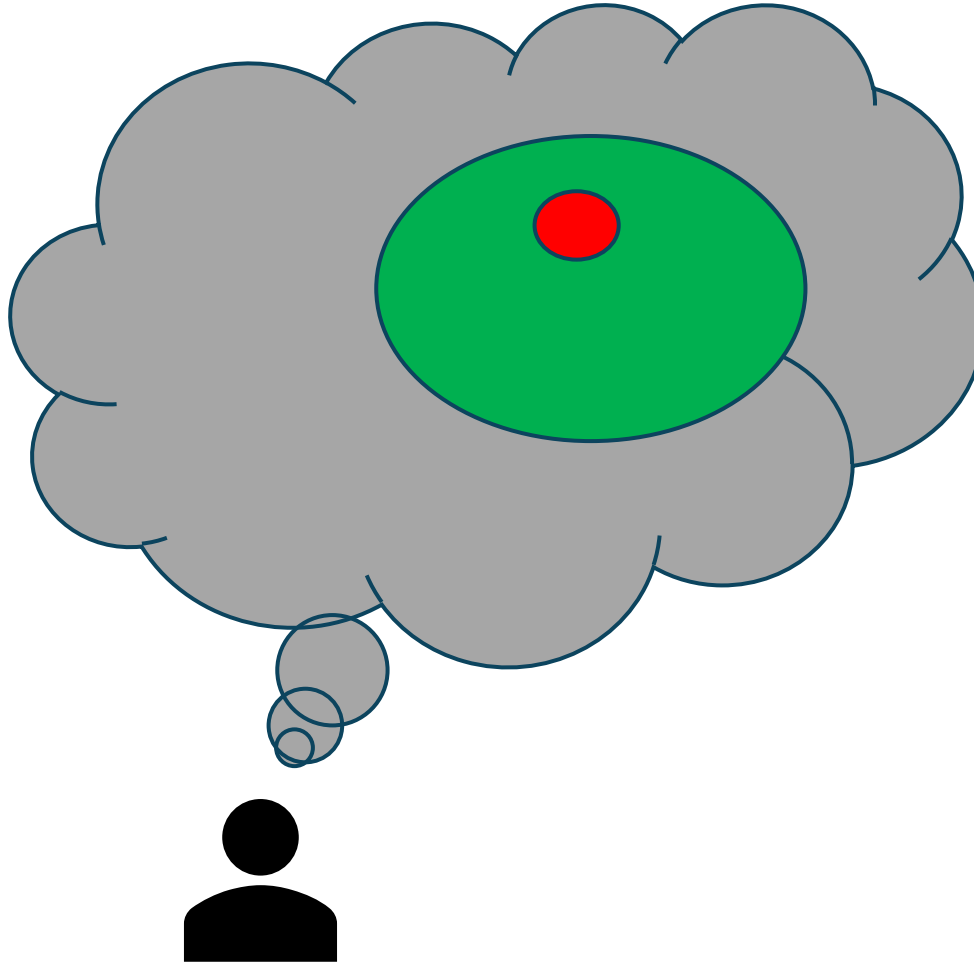
Version control information

The screenshot displays the Code Compass web interface, which is used for managing and viewing version control information for code files. The interface is divided into several sections:

- Search and Filter:** Includes a search bar with a dropdown menu set to "Text search", a "Settings" button, and a search expression input field. Below this, there are filters for "File Filter" (File name filter regex (*.cpp)) and "Directory Filter" (Path filter regex (click on a dir below)).
- File Manager:** A sidebar on the left showing a tree view of the project structure. The selected file is `cppservice.cpp` in the `./cppservice/src` directory. Other files listed include `Project: compass`, `Jump to sourcedir`, `... .libs`, `cppservice.cpp`, `cppservice.h`, `plugin.cpp`, `webplugin_cppservice_la-cppservice.d`, `webplugin_cppservice_la-cppservice.lo`, `webplugin_cppservice_la-cppservice.o`, `webplugin_cppservice_la-plugin.d`, `webplugin_cppservice_la-plugin.lo`, and `webplugin_cppservice_la-plugin.o`.
- Query Results:** A table showing the history of changes to the selected file. The table has columns for line number, author, date, and time. The changes are listed as follows:

Line	Author	Date	Time
13	Zoltán Borók-Nagy	2013-10-14	16:52
14	Zoltán Borók-Nagy	2013-07-30	15:59
15	Krupp	2013-05-14	10:32
16	Krupp	2013-05-14	10:32
17	Zalán Szügyi	2014-01-12	22:24
18	Zalán Szügyi	2014-01-12	22:24
19	Zalán Szügyi	2014-01-12	22:24
20	Zalán Szügyi	2014-01-12	22:24
21	Zalán Szügyi	2014-01-12	22:24
22	Zalán Szügyi	2014-01-12	22:24
23	Zalán Szügyi	2014-01-12	22:24
24	Krupp	2013-05-16	08:49
25	Krupp	2013-05-16	08:49
26	Zoltán Borók-Nagy	2013-05-09	10:06
27	Krupp	2013-05-16	08:49
28	Krupp	2013-05-16	08:49
29	Krupp	2013-05-16	08:49
30	Krupp	2013-05-16	08:49
31	Zalán Szügyi	2014-01-12	22:24
32	Krupp	2013-05-16	08:49
33	Zoltán Borók-Nagy	2013-05-09	10:06
34	Zalán Szügyi	2014-01-12	22:24
35	Krupp	2013-05-16	08:49
36	Zoltán Borók-Nagy	2013-05-09	10:06
37	Krupp	2013-05-16	08:49
38	Tibor Brunner	2015-02-17	12:52
39	Tibor Brunner	2015-02-17	12:52
40	Tibor Brunner	2015-02-17	12:52
41	Tibor Brunner	2015-02-17	12:52
42	Tibor Brunner	2015-02-17	12:52
43	Tibor Brunner	2015-02-17	12:52
44	Zoltán Borók-Nagy	2013-07-09	13:20
45	Zalán Szügyi	2014-01-12	22:24
46	Zoltán Borók-Nagy	2013-07-09	13:20
47	Zoltán Borók-Nagy	2013-07-09	13:20
48	Zoltán Borók-Nagy	2013-07-09	13:20
49	Zoltán Borók-Nagy	2013-07-09	13:20
50	Zoltán Borók-Nagy	2013-07-09	13:20
51	Krupp	2013-05-16	08:49
52	Zalán Szügyi	2014-01-12	22:24
- Find in this file:** A search bar for finding specific text within the file. The search expression is `(Use /re/ syntax for regexp search)`. The results are displayed in a table with columns for line number, author, date, and time.
- Code Editor:** The main area showing the source code of the selected file. The code is a C++ file named `cppservice.cpp`. It includes a header `"model/cxx/cppastnode.h"` and defines a `WORKSPACE_ID` constant. The code is organized into namespaces: `cc`, `service`, and `language`. The `language` namespace contains a `CppServiceHandler` class with methods `getAstNodeInfoByPosition`, `getInfoBox`, `getInfoBoxByPosition`, and `getAstNodeInfo`. The `getAstNodeInfoByPosition` method is highlighted, and a tooltip shows its description: "Description: const std::vector & filters".
- Context Buttons:** A panel on the right side of the code editor containing buttons for "File Outline", "File diagrams", and "Metrics".

Typical workflow



- Fast feature location
 - Textual search
 - Log-base search
- Extend knowledge
 - Info tree
 - CodeBites view
 - Function call chains
 - Browsing history
- **Validate knowledge**
 - All references
 - Architectural info
 - Version control info

Static Analysis / CodeChecker

Search: Text search Settings Search expression, like "foo AND bar" (see question mark icon for help)

File Filter: File name filter regex (*.cpp) Directory Filter: Path filter regex (click on a dir below)

whisperity/CodeCompass/llvm/lib/MC/MCDisassembler/Disassembler.cpp, line: 72

(Fully parsed) Disassembler.cpp :: /ssd/whisperity/CodeCompass/llvm/lib/MC/MCDisassembler/Disassembler.cpp

Find in this file: (Use /re/ syntax for regexp search) prev next

33 // Disassembler context. If null, it returns null.

34 //

35 LLVMDisasmContextRef

Entered call from 'LLVMCreateDisasmCPU'

36 LLVMCreateDisasmCPUFeatures(const char *TT, const char *CPU,

37 const char *Features, void *DisInfo, int TagType,

38 LLVMOpInfoCallback GetOpInfo,

39 LLVMSymbolLookupCallback SymbolLookup) {

40 // Get the target.

41 std::string Error;

42 const Target *TheTarget = TargetRegistry::lookupTarget(TT, Error);

43 if (!TheTarget)

Assuming 'TheTarget' is non-null

44 return nullptr;

45

46 const MCRegisterInfo *MRI = TheTarget->createMCRegInfo(TT);

47 if (!MRI)

Assuming 'MRI' is non-null

48 return nullptr;

49

50 // Get the assembler info needed to setup the MCContext.

51 const MCAsmInfo *MAI = TheTarget->createMCAsmInfo(*MRI, TT);

52 if (!MAI)

Assuming 'MAI' is non-null

53 return nullptr;

54

55 const MCInstrInfo *MII = TheTarget->createMCInstrInfo();

56 if (!MII)

Assuming 'MII' is non-null

57 return nullptr;

58

59 const MCSubtargetInfo *STI =

60 TheTarget->createMCSubtargetInfo(TT, CPU, Features);

61 if (!STI)

Assuming 'STI' is non-null

62 return nullptr;

63

64 // Set up the MCContext for creating symbols and MCEXPR's.

65 MCContext *Ctx = new MCContext(MAI, MRI, nullptr);

Memory is allocated

Reparse

File Manager /home/whisperity/Dummy

Project:

..

a.out

main.cpp

Search Results

Info Tree

```
TranslationUnitDecl 0x12eb718 <<invalid sloc>> <invalid sloc>
| -TypeDefDecl 0x12ebcd0 <<invalid sloc>> <invalid sloc> implicit __int128_t '__int128'
|   | -BuiltinType 0x12eb9b0 '__int128'
| -TypeDefDecl 0x12ebd38 <<invalid sloc>> <invalid sloc> implicit __uint128_t 'unsigned __int128'
|   | -BuiltinType 0x12eb9d0 'unsigned __int128'
| -TypeDefDecl 0x12ec078 <<invalid sloc>> <invalid sloc> implicit __NSConstantString '__NSConstantString_tag'
|   | -RecordType 0x12ebe20 '__NSConstantString_tag'
|   |   | -CXXRecord 0x12ebd88 '__NSConstantString_tag'
| -TypeDefDecl 0x12ec110 <<invalid sloc>> <invalid sloc> implicit __builtin_ms_va_list 'char *'
|   | -PointerType 0x12ec0d0 'char *'
|   |   | -BuiltinType 0x12eb7b0 'char'
| -TypeDefDecl 0x1323490 <<invalid sloc>> <invalid sloc> implicit __builtin_va_list '__va_list_tag [1]'
|   | -ConstantArrayType 0x12ec3f0 '__va_list_tag [1]' 1
|   |   | -RecordType 0x12ec200 '__va_list_tag'
|   |   |   | -CXXRecord 0x12ec160 '__va_list_tag'
| -FunctionDecl 0x1323538 </home/whisperity/Dummy/main.cpp:3:1, col:7> col:5 used f 'int ()'
| -FunctionDecl 0x1323640 prev 0x1323538 <line:5:1, line:8:1> line:5:5 used f 'int ()'
|   | -CompoundStmt 0x1323718 <line:6:1, line:8:1>
|   |   | -ReturnStmt 0x1323700 <line:7:3, col:10>
|   |   |   | -IntegerLiteral 0x13236e0 <col:10> 'int' 2
| -FunctionDecl 0x1323758 <line:10:1, line:17:1> line:10:5 main 'int ()'
|   | -CompoundStmt 0x1324140 <line:11:1, line:17:1>
|   |   | -DeclStmt 0x1324050 <line:14:3, col:24>
|   |   |   | -VarDecl 0x1323870 <col:3, col:23> col:8 used p 'int *' cinit
|   |   |   |   | -CXXNewExpr 0x1323ff8 <col:12, col:23> 'int *' Function 0x1323a38 'operator new' 'void *(unsigned long)'
|   |   |   |   |   | -CallExpr 0x1323990 <col:20, col:22> 'int'
|   |   |   |   |   |   | -ImplicitCastExpr 0x1323978 <col:20> 'int (*)()' <FunctionToPointerDecay>
|   |   |   |   |   |   | -DeclRefExpr 0x1323928 <col:20> 'int ()' lvalue Function 0x1323640 'f' 'int ()'
|   |   | -ReturnStmt 0x1324128 <line:16:3, col:15>
|   |   |   | -BinaryOperator 0x1324100 <col:10, col:15> 'int' '- '
|   |   |   |   | -ImplicitCastExpr 0x13240e8 <col:10, col:11> 'int' <LValueToRValue>
|   |   |   |   |   | -UnaryOperator 0x13240a8 <col:10, col:11> 'int' lvalue prefix '*' cannot overflow
|   |   |   |   |   |   | -ImplicitCastExpr 0x1324090 <col:11> 'int *' <LValueToRValue>
|   |   |   |   |   |   | -DeclRefExpr 0x1324068 <col:11> 'int *' lvalue Var 0x1323870 'p' 'int *'
|   |   |   |   |   |   | -IntegerLiteral 0x13240c8 <col:15> 'int' 2
| -FunctionDecl 0x1323a38 <<invalid sloc>> <invalid sloc> implicit used operator new 'void *(unsigned long)'
|   | -ParmVarDecl 0x1323b20 <<invalid sloc>> <invalid sloc> implicit 'unsigned long'
|   | -VisibilityAttr 0x1323ad8 <<invalid sloc>> Implicit Default
| -FunctionDecl 0x1323b88 <<invalid sloc>> <invalid sloc> implicit operator new[] 'void *(unsigned long)'
|   | -ParmVarDecl 0x1323c70 <<invalid sloc>> <invalid sloc> implicit 'unsigned long'
|   | -VisibilityAttr 0x1323c28 <<invalid sloc>> Implicit Default
| -FunctionDecl 0x1323d58 <<invalid sloc>> <invalid sloc> implicit used operator delete 'void (void *) noexcept'
|   | -ParmVarDecl 0x1323e40 <<invalid sloc>> <invalid sloc> implicit 'void *'
|   | -VisibilityAttr 0x1323df8 <<invalid sloc>> Implicit Default
| -FunctionDecl 0x1323ea8 <<invalid sloc>> <invalid sloc> implicit operator delete[] 'void (void *) noexcept'
|   | -ParmVarDecl 0x1323f90 <<invalid sloc>> <invalid sloc> implicit 'void *'
|   | -VisibilityAttr 0x1323f48 <<invalid sloc>> Implicit Default
```



Generated special methods

The screenshot displays an IDE interface with two main panels. The left panel, titled 'File Manager ..\parser\logger', shows the 'Info Tree' for the 'LogCommand' class. The tree structure is as follows:

- Type: LogCommand
 - Name: LogCommand
 - Qualified Name: LogCommand
 - Defined: logtoxml.cpp:18:1
 - Aliases
 - Inherits From
 - Inherited By
 - Friends
 - Methods
 - Compiler-generated
 - public
 - LogCommand()
 - LogCommand(const LogCommand &)
 - LogCommand(LogCommand &&)
 - operator=(LogCommand &&): LogCommand &
 - ~LogCommand()
 - Inherited
 - Members
 - Usage

The right panel shows the source code of 'logtoxml.cpp'. The code defines the 'LogCommand' class with several methods, including a constructor, a copy constructor, an assignment operator, and a destructor. The code is as follows:

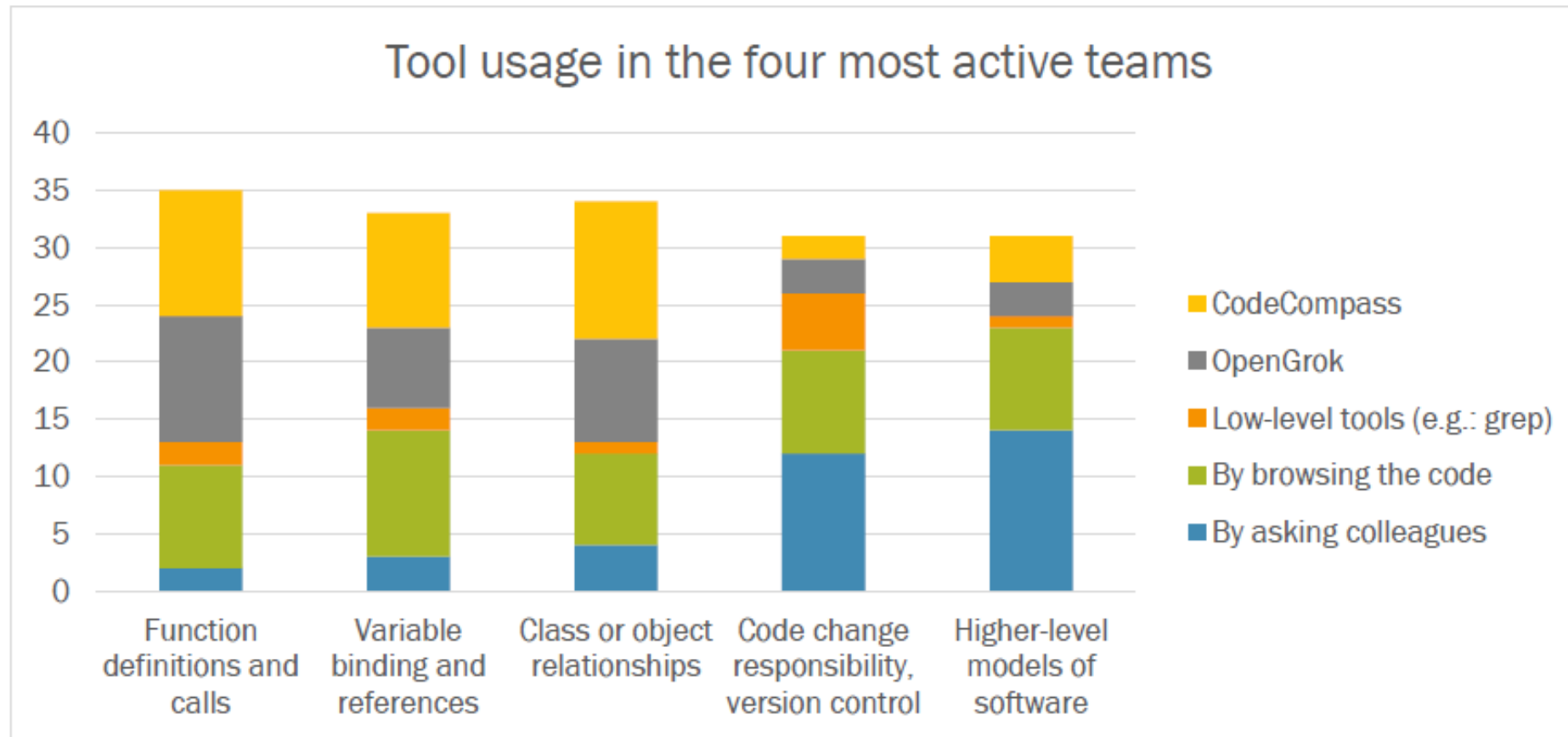
```
(Fully parsed) logtoxml.cpp :: /ssd/whisperity/CodeCompass_Discovery/ccSrcMaster/parser/logger/logtoxml.cpp
Find in this file: (Use /re/ syntax for regexp search) < prev next >
12 #include <boost/program_options.hpp>
12 #include <boost/algorithm/string.hpp>
Code Documentation
(Fully parsed) logtoxml.cpp :: /ssd/whisperity/CodeCompass_Discovery/ccSrcMaster/parser/logger/logtoxml.cpp
21
22 LogCommand::LogCommand()
23 : compiler()
24 , objects()
25 , params()
26 {}
27
28 LogCommand::LogCommand(const LogCommand & arg_1)
29 : compiler(arg_1.compiler)
30 , objects(arg_1.objects)
31 , params(arg_1.params)
32 {}
33
34 LogCommand::LogCommand(LogCommand && arg_1)
35 : compiler(std::move( arg_1.compiler ))
36 , objects(std::move( arg_1.objects ))
37 , params(std::move( arg_1.params ))
38 {}
39
40 LogCommand & LogCommand::operator=(LogCommand && arg_1)
41 {
42     this->compiler.operator=(std::move( arg_1.compiler ));
43     this->objects.operator=(std::move( arg_1.objects ));
44     this->params.operator=(std::move( arg_1.params ));
45     return *this;
46 }
47
48
49 LogCommand::~LogCommand()
50 {}
51
52
53
54
55
```

Experiences with CodeCompass

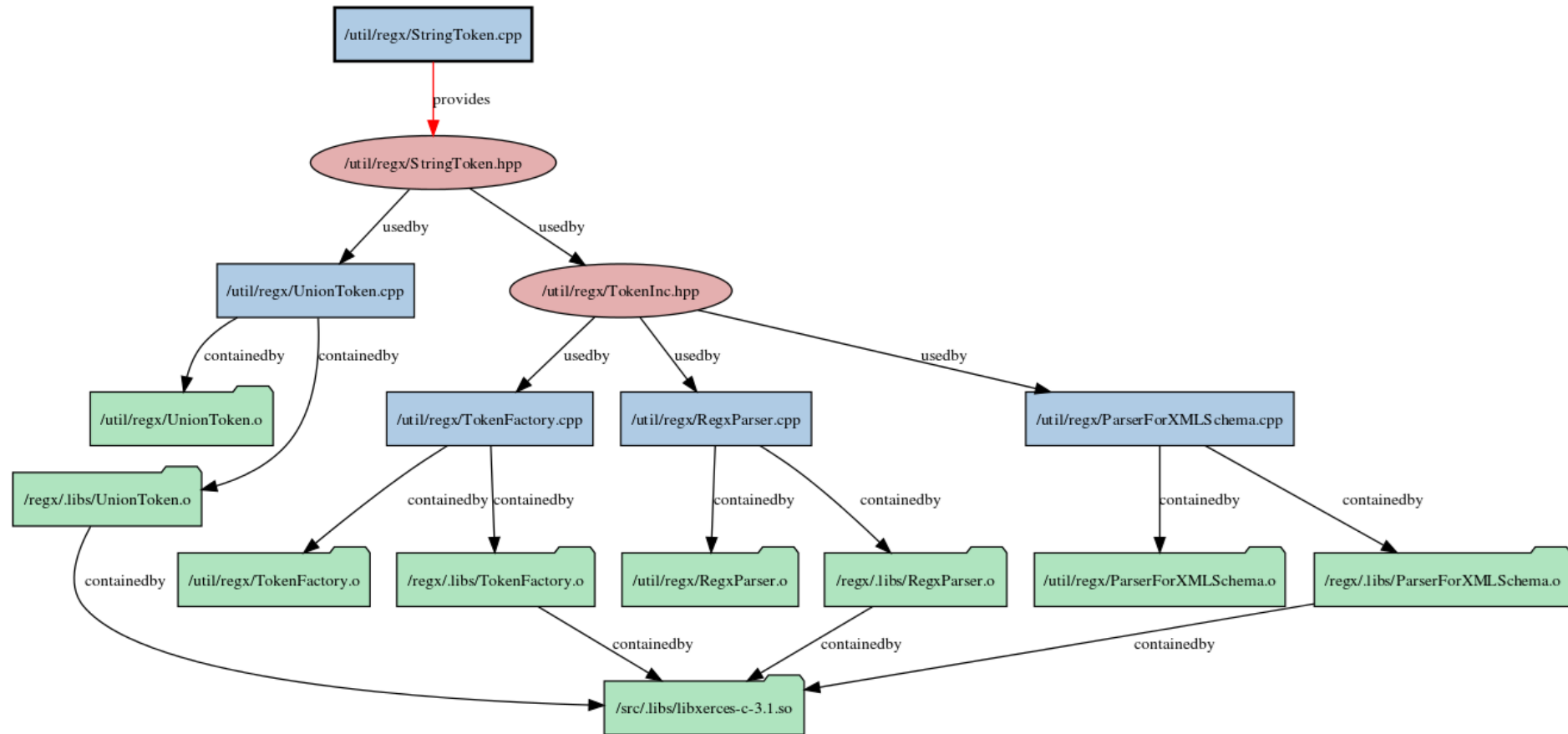
- Open source since Summer 2016
- Inside Ericsson
 - Voluntary based: no policy to enforce using CodeCompass
 - Replacing/extending openGrok and others
 - Frequently used together with CodeChecker
 - ~40 million lines of code parsed inside Ericsson
 - ~300-400 users
 - ~10% is very active
- A few known users outside of Ericsson
 - We intentionally do not collect information about users



Experiences with CodeCompass



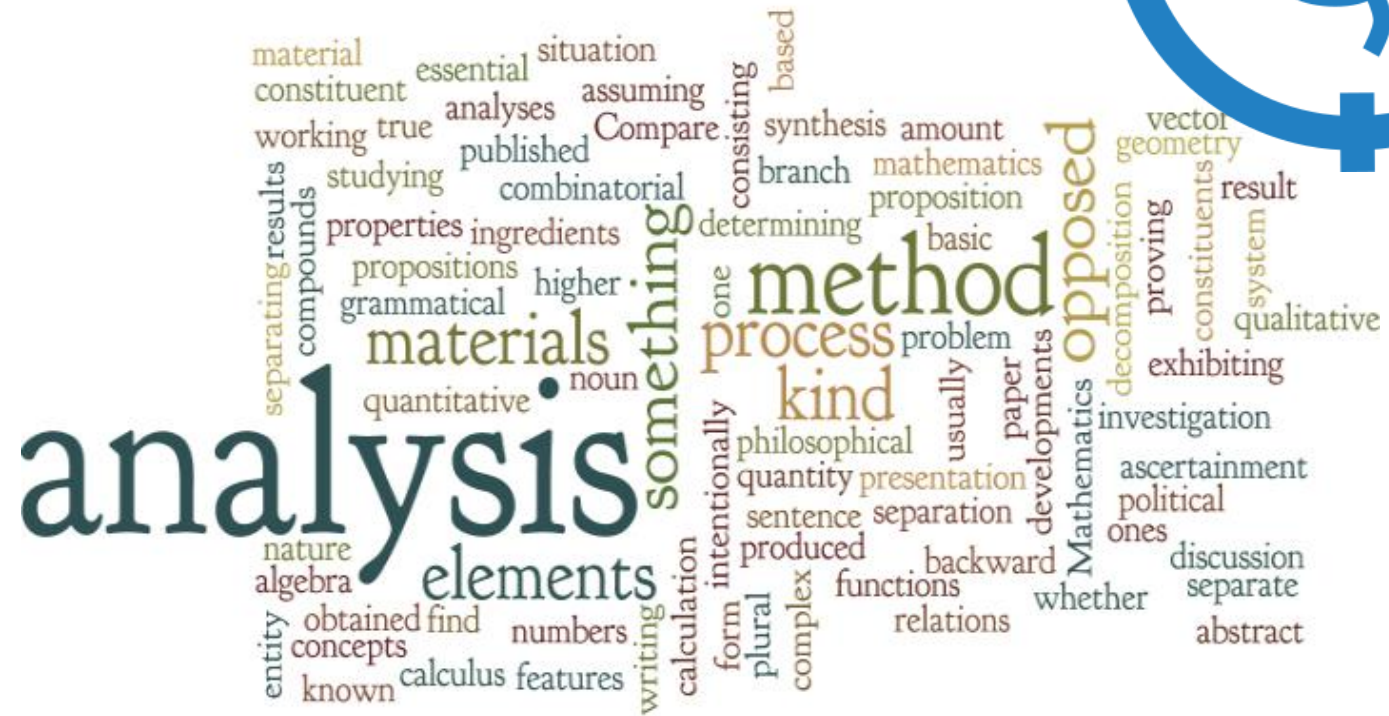
Experiences with CodeCompass



Static analysis

Analyzing the source without executing it

- Finding potential errors
- Optimization
- Automatic code refactoring
- Software metrics
- Code comprehension
- Many other tasks...



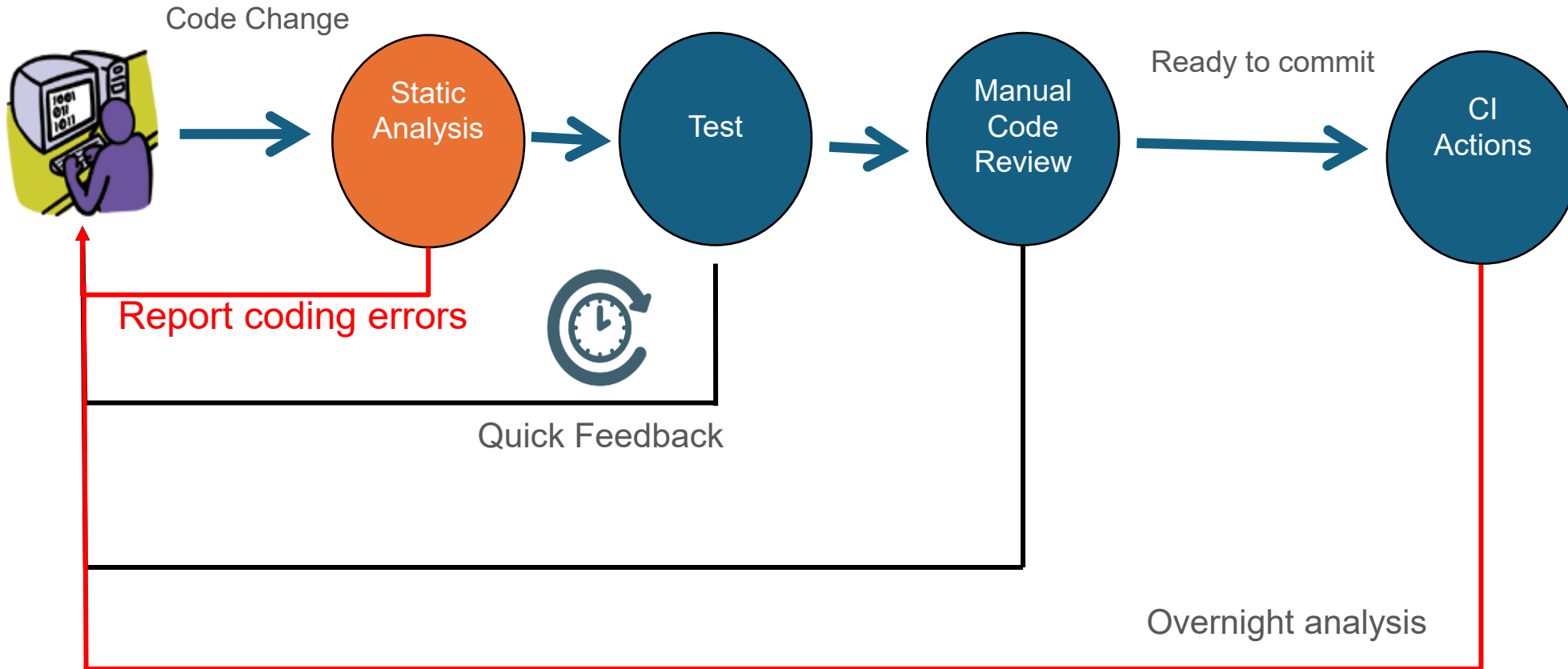
Finding bugs with static analysis

- All methods are using heuristics
- Over- or underestimate program behavior
- Neither sound nor complete
- False negatives and false positives

The biggest problem is the false positive reports



Finding bugs with static analysis



Rigorous, longer static analysis run daily (e.g. CTU)

Bug-finding methods

- Regular expression matching
- AST-based
- Symbolic execution
- Concolic execution
- Data flow analysis



Pattern matching

- Early versions of CppCheck
- Transform source to canonical format
- Apply rules in form of regular expression
- Context free -- weak detection capability
- Very low false positive rate

```
void f()
{
    int t[10];
    :
    t[10] = 0;    // array bound error
}
```

```
void g()
{
    int *ip;
    :
    ip = (int*) malloc(sizeof(ip));    // found
    ip = (int*) malloc(sizeof(double)); // not found
}
```



AST-based matching

- Most of the „Linter” tools, e.g. Clang Tidy
- Constructs the Abstract Syntax Tree (AST)
- Find patterns and then discover the environment
- Context sensitive, but not path sensitive
- Algorithmic complexity: $O(|\text{branches}|)$

```
#define ZERO 0
int f() { return 0; }
int g(int b)
{
    int a;
    switch(b)
    {
        case 1: a = b/0;      break;  // found
        case 2: a = b/ZERO; break;  // found
        case 3: a = 10/(b-3); break; // not found
        case 4: a = 10/f();   break; // not found
    }
    return a;
}
```



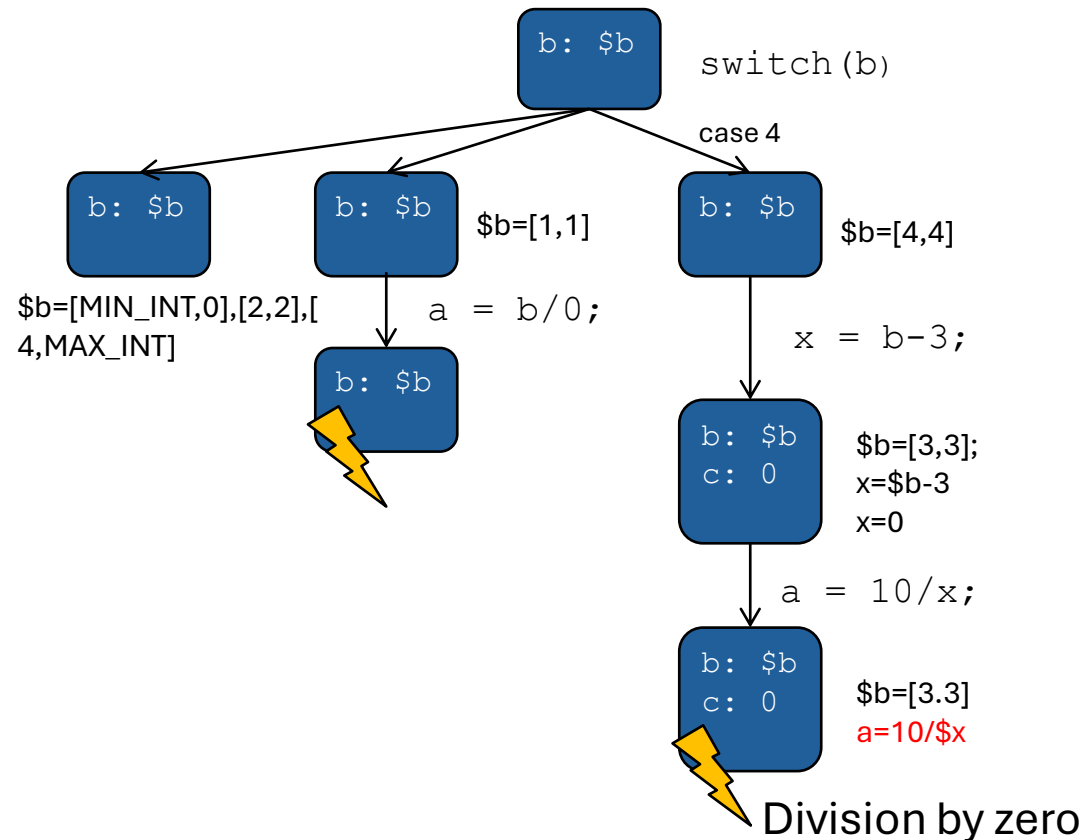
Symbolic execution

- Most advanced tools, e.g. Clang SA, Coverity, etc.
- Interprets the code on all possible execution paths
- Use symbolic values/intervals to represent variables
- Known functions are inlined
- Use SAT solver to eliminate impossible paths
- Context sensitive, path sensitive
- Algorithmic complexity: $O(2^{|branches|})$
- Mostly for existency check, not universal properties

```
#define ZERO 0
int f() { return 0; }
int g(int b)
{
    int a;
    switch(b)
    {
        case 1: a = b/0;      break;    // found
        case 2: a = b/ZERO; break;    // found
        case 3: a = 10/(b-3); break;  // not found
        case 4: a = 10/f();   break;  // not found
    }
    return a;
}
```



Symbolic execution



```

#define ZERO 0
int f() { return 0; }
int g(int b)
{
    int a;
    switch(b)
    {
        case 1: a = b/0;      break;    // found
        case 2: a = b/ZERO; break;    // found
        case 3: a = 10/(b-3); break;  // not found
        case 4: a = 10/f();  break;  // not found
    }
    return a;
}
    
```

Bug-finding methods

- Regular expression matching
- AST-based
- Symbolic execution
- Concolic execution
- Data flow analysis

The more methods and the more tools we use the more error we catch



Bug-finding methods

- Regular expression matching
- AST-based
- Symbolic execution
- Concolic execution
- Data flow analysis

The more methods and the more tools we use the more error we catch
... but the more burden is on the developer



What the developers want from static analysis tools

The top 3 issue

- Easy configurability
- Clear reports
- Less false positives



CodeChecker

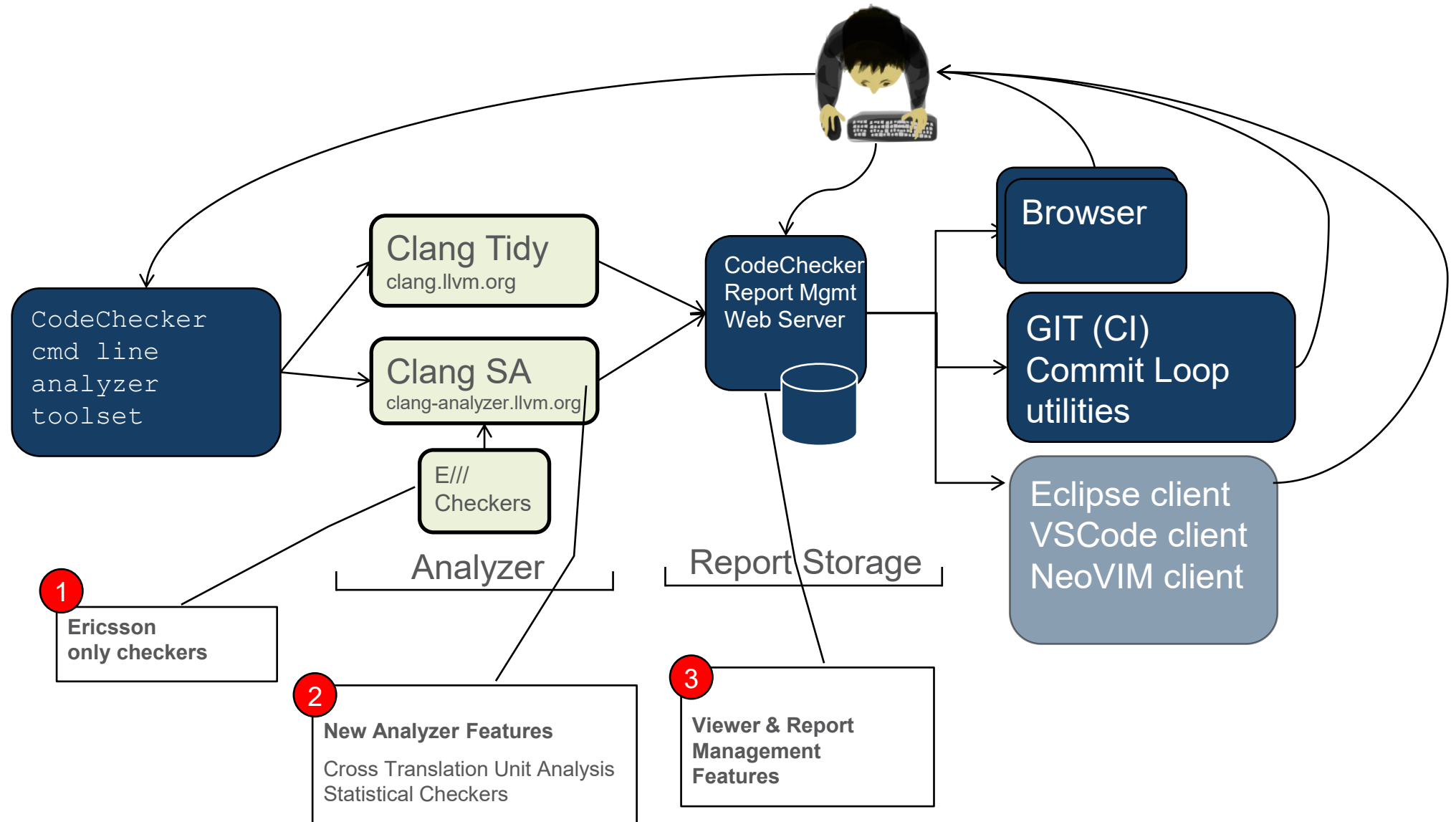
- Open-source static analysis framework in Python
- Executes multiply analyzers in a universal parametrisation
- Collects and stores the reports in a database
- Unify the findings*
- Filter, compare, present and manage findings
- Annotate the findings
- Easy to integrate into CI/CD pipelines

<https://github.com/ericsson/codechecker>

*almost done



CodeChecker architecture



CodeChecker usage

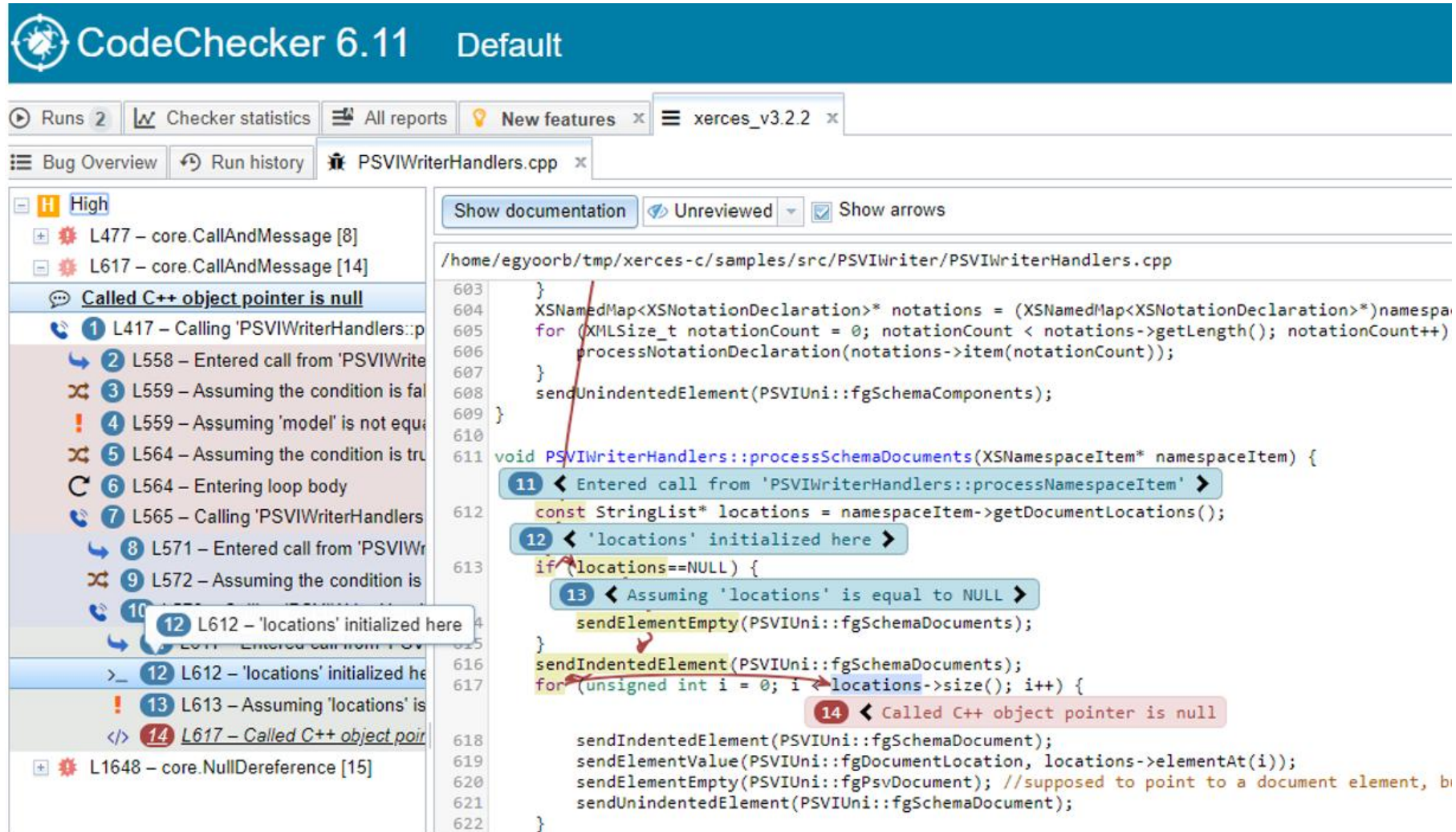
The screenshot displays the CodeChecker 6.13 web interface for the Xerces project. The top navigation bar includes links for PRODUCTS, RUNS, RUN HISTORY, STATISTICS, and REPORTS. The left sidebar contains filters for Diff type, File path, Checker name, Severity, Review Status, Detection status, Source component, Checker message, and Detection date. A dropdown menu is open for the Severity filter, showing the following options:

- ☐ Unspecified (0)
- ☐ Style (0)
- ☐ Low (50)
- ☒ Medium (49)
- ☐ High (46)
- ☐ Critical (0)

The main table displays a list of detected issues with the following columns: Report hash, File, Message, Checker name, Severity, Bug path length, Review status, and Detection status. The table shows several issues related to the 'xerces-c' project, including warnings about missing constructors and assignment operators, and deadcode warnings for 'duration', 'startSrc', 'key', and 'wasAdded'.

Report hash	File	Message	Checker name	Severity	Bug path length	Review status	Detection status
bd22fe8415...	xerces-c-3.1.0/src/xercesc/util/Hash2KeysSetOf.hpp	class 'Hash2KeysSetOf' defines a non-default destructor, a copy constructor and a copy assignment operator but does not define a move constructor or a move assignment operator	cppcoreguidelines-special-member-functions	L	1	🔗	🚨
741228491d...	xerces-c-3.1.0/src/xercesc/util/Hash2KeysSetOf.hpp	class 'Hash2KeysSetOfEnumerator' defines a non-default destructor, a copy constructor and a copy assignment operator but does not define a move constructor or a move assignment operator	cppcoreguidelines-special-member-functions	L	1	🔗	🚨
62316331bf...	xerces-c-3.1.0/samples/src/PSVWriter/PSVWriter.cpp	Value stored to 'duration' is never read	deadcode_DeadStores	L	1	🔗	🚨
	xerces-c-	class 'StrX' defines a non-default destructor but does not define a copy constructor, a copy assignment operator, a move constructor or a move assignment operator	cppcoreguidelines-special-member-functions	L	1	🔗	🚨
	xerces-c-	class 'DOMLSSerializer' defines a non-default destructor, a copy constructor and a copy assignment operator but does not define a move constructor or a move assignment operator	cppcoreguidelines-special-member-functions	L	1	🔗	🚨
		Value stored to 'startSrc' during its initialization is never read	deadcode_DeadStores	L	1	🔗	🚨
		Value stored to 'key' is never read	deadcode_DeadStores	L	1	🔗	🚨
		Value stored to 'wasAdded' is never read	deadcode_DeadStores	L	1	🔗	🚨
	ecvRedirect.hpp	class 'StrX' defines a non-default destructor but does not define a copy constructor, a copy assignment operator, a move constructor or a move assignment operator	cppcoreguidelines-special-member-functions	L	1	🔗	🚨
0d512c5dc9...	xerces-c-3.1.0/samples/src/RedirectHandlers.hpp	class 'RedirectHandlers' defines a non-default destructor but does not define a copy constructor, a copy assignment operator, a move constructor or a move assignment operator	cppcoreguidelines-special-member-functions	L	1	🔗	🚨
5b940b6578...	xerces-c-3.1.0/samples/src/SCMPrint/SCMPrint.cpp	class 'StrX' defines a non-default destructor but does not define a copy constructor, a copy assignment operator, a move constructor or a move assignment operator	cppcoreguidelines-special-member-functions	L	1	🔗	🚨
	xerces-c-	class 'SCMPrintHandler' defines a non-default destructor but does					

CodeChecker usage



The screenshot displays the CodeChecker 6.11 interface. The top bar shows the application name and version. Below it, there are tabs for 'Runs', 'Checker statistics', 'All reports', 'New features', and a file tab for 'xerces_v3.2.2'. The main area is divided into a left sidebar for 'Bug Overview' and 'Run history', and a central pane for the source code file 'PSVIWriterHandlers.cpp'.

The left sidebar shows a list of bugs. The selected bug is 'Called C++ object pointer is null' (L617). The list includes:

- L477 - core.CallAndMessage [8]
- L617 - core.CallAndMessage [14]
- L417 - Calling 'PSVIWriterHandlers::p' (selected)
- L558 - Entered call from 'PSVIWrite'
- L559 - Assuming the condition is fal
- L559 - Assuming 'model' is not equ
- L564 - Assuming the condition is tru
- L564 - Entering loop body
- L565 - Calling 'PSVIWriterHandlers'
- L571 - Entered call from 'PSVIWr
- L572 - Assuming the condition is
- L612 - 'locations' initialized here (selected)
- L613 - Assuming 'locations' is
- L617 - Called C++ object poin
- L1648 - core.NullDereference [15]

The central pane shows the source code for 'PSVIWriterHandlers.cpp'. The code is annotated with numbered callouts (1-14) corresponding to the bugs in the sidebar. A red line highlights the 'locations' variable initialization and its subsequent use in a loop. The code includes headers for 'XSNamespaceItem' and 'XSNotationDeclaration'.

```
603 }
604 XSNamespaceItem* notations = (XSNamespaceItem*)namespaceItem;
605 for (XMLSize_t notationCount = 0; notationCount < notations->getLength(); notationCount++)
606     processNotationDeclaration(notations->item(notationCount));
607 }
608 sendUnindentedElement(PSVIUni::fgSchemaComponents);
609 }
610
611 void PSVIWriterHandlers::processSchemaDocuments(XSNamespaceItem* namespaceItem) {
612     11 < Entered call from 'PSVIWriterHandlers::processNamespaceItem' >
613     const StringList* locations = namespaceItem->getDocumentLocations();
614     12 < 'locations' initialized here >
615     if (locations==NULL) {
616         13 < Assuming 'locations' is equal to NULL >
617         sendElementEmpty(PSVIUni::fgSchemaDocuments);
618     }
619     sendIndentedElement(PSVIUni::fgSchemaDocuments);
620     for (unsigned int i = 0; i < locations->size(); i++) {
621         14 < Called C++ object pointer is null
622         sendIndentedElement(PSVIUni::fgSchemaDocument);
623         sendElementValue(PSVIUni::fgDocumentLocation, locations->elementAt(i));
624         sendElementEmpty(PSVIUni::fgPsvDocument); //supposed to point to a document element, b
625         sendUnindentedElement(PSVIUni::fgSchemaDocument);
626     }
```

CodeChecker users

Industry

- Ericsson
- Google running on Chromium, Fuchsia
- Mozilla running on Firefox
- Electronic Arts
- BMW
- Sony
- Samsung

Academy

- Wright University, Ohio, US -- Secure software lab curriculum



CodeChecker users

Gitlab

- GitLab Advanced SAST (Static Application Security Testing)
- Wrapped as a Docker image
- Pre-installed and integrated to the default SAST configuration template



CodeChecker success factors

Build integration

- Strong integration to the build system (Cmake, Bazel or LD_PRELOAD based)
- Using the newest C/C++ parser
- Storing the result in database and query it in various ways
- Strong visualization
- Tool agnostic/Easy integration of new tools
- Reacting the user feedback



Some interesting development

- Cross-translational unit checks
- Statistical checks
- Modelling standard library lifetime (e.g., `std::string` and `std::string_view` class)
- Detecting misuse of tagged-unions and `std::variant` classes
- Improving the taint analysis
- Detecting regular expression misuses



Static analysis software lab

- Academia-industry co-operation with Ericsson, Bosh, Erlang Solution, OTP Bank, etc.
 - Students work on industry-motivated open-source projects
 - Supervised by PhD students and industry developers
 - BSc, MSc and PhD students can participate, MSc students can receive credits
 - Students learn soft- and hard skills, participation in open-source projects, reacting the reviews
 - Source of many BSc and MSc theses, academic papers, PhD theses
 - Many students find their first job at the industrial partners.
-
- CodeCompass now is 100% developed and maintained by software lab students
 - CodeChecker has a huge student involvement of checker development



Future challenges

- Improve Clang Static Analyzer engine
- Further reduce false positives
- Better taint analysis
- Better integrate data flow analysis into current methods
- Further improve warning quality
- AI integration (?)



References

- [1] D. Farago, F. Merz, C. Sinz:
[Automatic Heavy-weight Static Analysis Tools for Finding Bugs in Safety-critical Embedded C/C++ Code](#)
- [2] Par Emanuelsson and Ulf Nilsson, A Comparative Study of Industrial Static Analysis Tools
- [3] G. Horvath, [Make Friends with the Clang Static Analysis tools](#), CppCon 2016
- [4] CodeChecker: <https://github.com/Ericsson/codechecker>
- [5] Clang: <http://clang.llvm.org/>
- [6] Clang Static Analyzer: <http://clang-analyzer.llvm.org/>
- [7] Clang Tidy: <http://clang.llvm.org/extra/clang-tidy/>
- [9] CodeChecker on GITHUB: <https://github.com/Ericsson/codechecker>
- [10] CodeChecker on OpenALM: <https://openalm.lmera.ericsson.se/projects/codechecker>
- [11] CodeCompass: <https://github.com/Ericsson/codecompass>
- [12] CodeCompass trial site: <https://codecompass.net>
- [13] G. Márton, D. Krupp: Tool talk: CodeChecker <https://www.youtube.com/watch?v=bVqrhaoxHlc>, PLDI 2020



Industrial experiences in the use of static analysis

Zoltán Porkoláb

gsd@inf.elte.hu

zoltan.porkolab@ericsson.com

<https://gsd.web.elte.hu>



<https://github.com/Ericsson/codecompass>



<https://github.com/Ericsson/codechecker>

